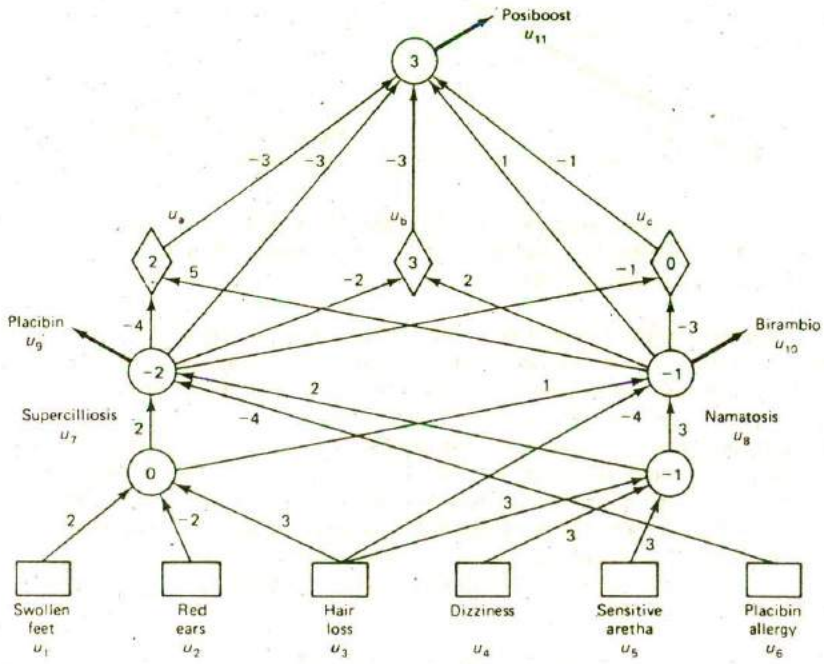

Figure 15.10 A simple neural network expert system. (From S. I. Gallant, ACM Communications, Vol. 31, No. 2, p. 152, 1988. By permission.)

of +1 (true). Negative symptoms are given an input value of −1 (false), and unknown symptoms are given the value 0. Input symptom values are multiplied by their corresponding weights w_{ij}. Numbers within the nodes are initial bias weights w_{j0}, and numbers on the links are the other node input weights. When the sum of the weighted products of the inputs exceeds 0, an output will be present on the corresponding node output and serve as an input to the next layer of nodes.

As an example, suppose the patient has swollen feet ($u_1 = +1$) but not red ears ($u_2 = -1$) nor hair loss ($u_3 = -1$). This gives a value of $u_7 = +1$ (since $0+(2)(1)+(-2)(-1)+(3)(-1) = 1$), suggesting the patient has superciliosis.

When it is also known that the other symptoms of the patient are false ($u_4 = u_5 = u_6 = -1$), it may be concluded that namatosis is absent ($u_8 = -1$), and therefore that birambio ($u_{10} = +1$) should be prescribed while placibin should not be prescribed ($u_9 = -1$). In addition, it will be found that posiboost should also be prescribed ($u_{11} = +1$).

The intermediate triangular shaped nodes were added by the training algorithm. These additional nodes are needed so that weight assignments can be made which permit the computations to work correctly for all training instances.

Deductions can be made just as well when only partial information is available. For example, when a patient has swollen feet and suffers from hair loss, it may be concluded the patient has superciliosis, regardless of whether or not the patient has red ears. This is so because the unknown variable cannot force the sum to change to negative.

A system such as this can also explain how or why a conclusion was reached. For example, when inputs and outputs are regarded as rules, an output can be explained as the conclusion to a rule. If placibin is true, the system might explain why with a statement such as

Placibin is TRUE due to the following rule:

IF Placibin Alergy (u_6) is FALSE, and
Superciliosis is TRUE

THEN Conclude Placibin is TRUE.

Expert systems based on neural network architectures can be designed to possess many of the features of other expert system types, including explanations for how and why, and confidence estimation for variable deduction.

15.4 DEALING WITH UNCERTAINTY

In Chapters 5 and 6 we discussed the problem of dealing with uncertainty in knowledge-based systems. We found that different approaches were possible including probabilistic (Bayesian or Shafer-Dempster), the use of fuzzy logic, ad-hoc, and heuristic methods. All of these methods have been applied in some form of system, and many building tools permit the use of more than a single method. The ad-hoc method used in MYCIN was described in Section 6.5. Refer to that section to review how uncertainty computations are performed in a typical expert system.

15.5 KNOWLEDGE ACQUISITION AND VALIDATION

One of the most difficult tasks in building knowledge-based systems is in the acquisition and encoding of the requisite domain knowledge. Knowledge for expert systems must be derived from expert sources like experts in the given field, journal articles, texts, reports, data bases, and so on. Elicitation of the right knowledge can take several man years and cost hundreds of thousands of dollars. This process is now recognized as one of the main bottlenecks in building expert and other knowledge-based systems. Consequently, much effort has been devoted to more effective methods of acquisition and coding.

Pulling together and correctly interpreting the right knowledge to solve a set

of complex tasks is an onerous job. Typically, experts do not know what specific knowledge is being applied nor just how it is applied in the solution of a given problem. Even if they do know, it is likely they are unable to articulate the problem solving process well enough to capture the low-level knowledge used and the inferring processes applied. This difficulty has led to the use of AI experts (called knowledge engineers) who serve as intermediaries between the domain expert and the system. The knowledge engineer elicits information from the experts and codes this knowledge into a form suitable for use in the expert system.

The knowledge elicitation process is depicted in Figure 15.11. To elicit the requisite knowledge, a knowlege engineer conducts extensive interviews with domain experts. During the interviews, the expert is asked to solve typical problems in the domain of interest and to explain his or her solutions.

Using the knowledge gained from the experts and other sources, the knowledge engineer codes the knowledge in the form of rules or some other representation scheme. This knowledge is then used to solve sample problems for review and validation by the experts. Errors and omissions are uncovered and corrected, and additional knowledge is added as needed. The process is repeated until a sufficient body of knowledge has been collected to solve a large class of problems in the chosen domain. The whole process may take as many as tens of person years.

Penny Nii, an experienced knowledge engineer at Stanford University, has described some useful practices to follow in solving acquisition problems through a sequence of heuristics she uses. They have been summarized in the book *The Fifth Generation* by Feigenbaum and McCorduck (1983) as follows.

> You can't be your own expert. By examining the process of your own expertise you risk becoming like the centipede who got tangled up in her own legs and stopped dead when she tried to figure out how she moved a hundred legs in harmony.
>
> From the beginning, the knowledge engineer must count on throwing efforts away. Writers make drafts, painters make preliminary sketches; knowledge engineers are no different.
>
> The problem must be well chosen. AI is a young field and isn't ready to take on every problem the world has to offer. Expert systems work best when the problem is well bounded, which is computer talk to describe a problem for which large amounts of specialized knowledge may be needed, but not a general knowledge of the world.
>
> If you want to do any serious application you need to meet the expert more than half way; if he's had no exposure to computing, your job will be that much harder.
>
> If none of the tools you normally use works, build a new one.
>
> Dealing with anything but facts implies uncertainty. Heuristic knowledge is not hard and fast and cannot be treated as factual. A weighting procedure has to be built into

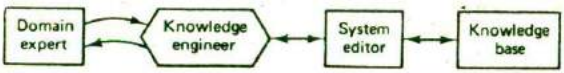

Figure 15.11 The knowledge acquisition process.

the expert system to allow for expressions such as "I strongly believe that . . ." or "The evidence suggests that. . . ."

A high-performance program, or a program that will eventually be taken over by the expert for his own use, must have very easy ways of allowing the knowledge to be modified so that new information can be added and out-of-date information deleted.

The problem needs to be a useful, interesting one. There are knowledge-based programs to solve arcane puzzles, but who cares? More important, the user has to understand the system's real value to his work.

When Nii begins a project, she first persuades a human expert to commit the considerable time that is required to have the expert's mind mined. Once this is done, she immerses herself in the given field, reading texts, articles, and other material to better understand the field and to learn the basic jargon used. She then begins the interviewing process. She asks the expert to describe his or her tasks and problem solving techniques. She asks the expert to choose a moderately difficult problem to solve as an example of the basic approach. This information is then collected, studied, and presented to other members of the development team so that a quick prototype can be constructed for the expert to review. This serves several purposes. First, it helps to keep the expert in the development loop and interested. Secondly, it serves as a rudimentary model with which to uncover flaws and other problems. It also helps both expert and developer in discovering the real way the expert solves problems. This usually leads to a repeat of the problem solving exercise, but this time in a step-by-step walk through of the sample problem. Nii tests the accuracy of the expert's explanations by observing his or her behavior and reliance on data and other sources of information. She is concerned more with the manipulation of the knowledge than with the actual facts. Keeping the expert focused on the immediate problem requires continual prompting and encouragement.

During the whole interview process Nii is mentally examining alternative approaches for the best knowledge representation and inferencing methods to see how well each would best match the expert's behavior. The whole process of elicitation, coding, and verification may take several iterations over a period of several months.

Recognizing the acquisition bottleneck in building expert systems, researchers and vendors alike have sought new and better ways to reduce the burden and reliance placed on knowledge engineers, and in general, ways to improve and speed up the development process. This has led to a number of sophisticated building tools which we consider next.

15.6 KNOWLEDGE SYSTEM BUILDING TOOLS

Since the introduction of the first successful expert systems in the late 1970s, a large number of building tools have been introduced, both by the academic community and industry. These tools range from high level programming languages to intelligent editors to complete shell environment systems. A number of commercial products

are now available ranging in price from a few hundred dollars to tens of thousands of dollars. Some are capable of running on medium size PCs while others require larger systems such as LISP machines, minis, or even main frames.

When evaluating building tools for expert system development, the developer should consider the following features and capabilities that may be offered in systems.

1. Knowledge representation methods available (rules, logic-based network structures, frames, with or without inheritance, multiple world views, object-oriented, procedural, and the methods offered for dealing with uncertainty, if any).
2. Inference and control methods available (backward chaining, forward chaining, mixed forward and backward chaining, blackboard architecture approach, logic driven theorem prover, object-oriented approach, the use of attached procedures, types of meta-control capabilities, forms of uncertainty management, hypothetical reasoning, truth maintenance management, pattern matching with or without indexing, user functions permitted, linking options to modules written in other languages, and linking options to other sources of information such as data bases).
3. User interface characteristics (editor flexibility and ease of use, use of menus, use of pop-up windows, developer provided text capabilities for prompts and help messages, graphics capabilities, consistency checking for newly entered knowledge, explanation of how and why capabilities, system help facilities, screen formatting and color selection capabilities, network representation of knowledge base, and forms of compilation available, batch or interactive).
4. General system characteristics and support available (types of applications with which the system has been successfully used, the base programming language in which the system was written, the types of hardware the systems are supported on, general utilities available, debugging facilities, interfacing flexibility to other languages and databases, vendor training availability and cost, strength of software support, and company reputation).

In the remainder of this section, we describe a few representative building tool systems. For a more complete picture of available systems, the reader is referred to other sources.

Personal Consultant Plus

A family of Personal Consultant expert system shells was developed by Texas Instruments, Inc. (TI) in the early 1980s. These shells are rule-based building tools patterned after the MYCIN system architecture and developed to run on a PC as well as on larger systems such as the TI Explorer. The largest and most versatile of the Personal Consultant family is Personal Consultant Plus.

Personal Consultant Plus permits the use of structures called frames (different

from the frames of Chapter 7) to organize functionally related production rules into subproblem groups. The frames are hierarchically linked into a tree structure which is traversed during a user consultation. For example, a home electrical appliance diagnostic system might have subframes of microwave cooker, iron, blender, and toaster as depicted in Figure 15.12.

When diagnosing a problem, only the relevant frames would be matched, and the rules in each frame would address only that part of the overall problem. A feature of the frame structure is that parameter property inheritance is supported from ancestor to descendant frames.

The system supports certainty factors both for parameter values and for complete rules. These factors are propagated throughout a chain of rules used during a consultation session, and the resultant certainty values associated with a conclusion are presented. The system also has an explanation capability to show how a conclusion was reached by displaying all rules that lead to a conclusion and why a fact is needed to fire a given rule.

An interactive dialog is carried out between user and system during a consultation session. The system prompts u[illegible] with English statements provided by the developer. Menu windows with selectable colors provide the user parameter value selections. A help facility is also available so the developer can provide explanations and give general assistance to the user during a consultation session.

A system editor provides a number of helpful facilities to build, store, and print a knowledge base. Parameter and property values, textual prompts, help messages, and debug traces are all provided through the editor. In addition, user defined functions written in LISP may be provided by a developer as part of a rule's conditions and/or actions. The system also provides access to other sources of information, such as dBase II and dBase III. An optional graphics package is also available at extra cost.

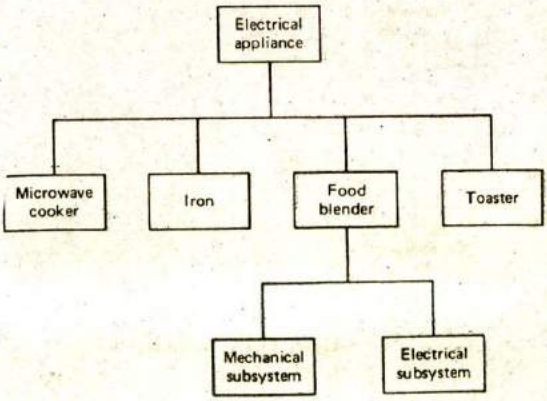

Figure 15.12 Hierarchical frame structure in PC Plus.

Radian Rulemaster

The Rulemaster system developed in the early 1980s by the Radian Corporation was written in C language to run on a variety of mini- and microcomputer systems. Rulemaster is a rule-based building tool which consists of two main components: Radial, a procedural, block structured language for expressing decision rules related to a finite state machine, and Rulemaker, a knowledge acquisition system which induces decision trees from examples supplied by an expert. A program in Rulemaster consists of a collection of related modules which interact to affect changes of state. The modules may contain executable procedures, advice, or data. The building system is illustrated in Figure 15.13.

Rulemaster's knowledge can be based on partial certainty using fuzzy logic or heuristic methods defined by the developer. Users can define their own data types or abstract types much the same as in Pascal. An explanation facility is provided to explain its chain of reasoning. Programs in other languages can also be called from Rulemaster.

One of the unique features of Rulemaster is the Rulemaker component which has the ability to induce rules from examples. Experts are known to have difficulty in directly expressing rules related to their decision processes. On the other hand, they can usually come up with a wealth of examples in which they describe typical solution steps. The examples provided by the expert offer a more accurate way in

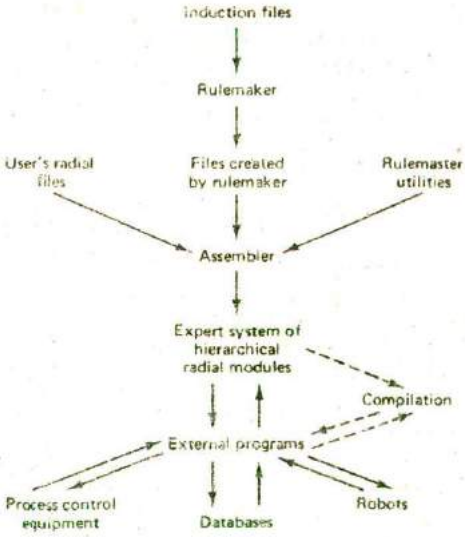

Figure 15.13 Rulemaster building system.

which the problem solving process is carried out. These examples are transformed into rules by Rulemaker through an induction process.

KEE (Knowledge Engineering Environment)

KEE is one of the more popular building tools for the development of larger-scale systems. Developed by Intellicorp, this system employs sophisticated representation schemes structured around frames called units. The frames are made up of slots and facets which contain object attribute values, rules, methods, logical assertions, text, or even other frames. The frames are organized into one or more knowledge bases in the form of hierarchical structures which permit multiple inheritance down hierarchical paths. Rules, procedures, and object oriented representation methods are also supported.

Inference is carried out through inheritance, forward-chaining, backward-chaining, or a mixture of these methods. A form of hypothetical reasoning is also provided through different viewpoints which may be explored concurrently. The viewpoints represent different aspects of a situation, views of the same situation taken at different times, hypothetical situations, or alternative courses of action. This feature permits a user to compare competing courses of action or to reason in parallel about partial solutions based on different approaches.

KEE's support environment includes a graphics-oriented debugging package, flexible end-user interfaces using windows, menus, and an explanation capability with graphic displays which can show inference chains. A graphics-based simulation package called SimKit is available at additional cost.

KEE has been used for the development of intelligent user interfaces, genetics, diagnosis and monitoring of complicated systems, planning, design, process control, scheduling, and simulation. The system is LISP based, developed for operation on systems such as Symbolics machines, Xerox 1100s, or TI Explorers. Systems can also be ported to open architecture machines which support Common LISP without extensive modification.

OPS5 System

The OPS5 and other OPS building tools were developed at Carnegie Mellon University in conjunction with DEC during the late 1970s. This system was developed to build the R1/XCON expert system which configures Vax and other DEC minicomputer systems. The system is used to build rule-based production systems which use forward chaining in the inference process (backward and mixed chaining is also possible). The system was written in C language to run on the DEC Vax and other minicomputers. It uses a sophisticated method of indexing rules (the Rete algorithm) to reduce the matching times during the match-select-execute cycle. Examples of OPS5 rules were given above in Section 15.2, and a description of the Rete match algorithm was given in Section 10.6.

15.7 SUMMARY

Expert and other knowledge-based systems are usually composed of at least a knowledge base, an inference engine, and some form of user interface. The knowledge base, which is separate from the inference and control components, contains the expert knowledge coded in some form such as production rules, networks of frames or other representation scheme. The inference engine manipulates the knowledge structures in the knowledge base to perform a type of symbolic reasoning and draw useful conclusions relating to the current task. The user interface provides the means for dialog between the user and system. The user inputs commands, queries, and responses to system messages, and the system, in turn, produces various messages for the user. In addition to these three components, most systems have an editor for use in creating and modifying the knowledge base structures and an explanation module which provides the user with explanations of how a conclusion was reached or why a piece of knowledge is needed. A few systems also have some learning capability and a case history file with which [illegible] complete consultation traces.

A variety of expert system architectures have been constructed including rule-based systems, frame-based systems, decision tree (discrimination network) systems, analogical reasoning systems, blackboard architectures, theorem proving systems, and even neural network architectures. These systems may differ in the direction of rule chaining, in the handling of uncertainty, and in the search and pattern matching methods employed. Rule and frame based systems are by far the most popular architectures used.

Since the introduction of the first expert systems in the late 1970s, a number of building tools have been developed. Such tools may be as unsophisticated as a bare high level language or as comprehensive as a complete shell development environment. A few representative building tools have been described and some general characteristics of tools for developers were given.

The acquisition of expert knowledge for knowledge-based systems remains one of the main bottlenecks in building them. This has led to a new discipline called knowledge engineering. Knowledge engineers build systems by eliciting knowledge from experts, coding that knowledge in an appropriate form, validating the knowledge, and ultimately constructing a system using a variety of building tools.

EXERCISES

15.1. What are the main advantages in keeping the knowledge base separate from the control module in knowledge-based systems?

15.2. Why is it important that an expert system be able to explain the why and how questions related to a problem solving session?

15.3. Give an example of the use of metaknowledge in expert systems inference.

15.4. Describe and compare the different types of problems solved by four of the earliest expert systems DENDRAL, MYCIN, PROSPECTOR, and R1.

15.5. Identify and describe two good application areas for expert systems within a university environment.

15.6. How do rules in PROLOG differ from general production system rules?

15.7. Make up a small knowledge-base of facts and rules using the same syntax as that used in Figure 15.2 except that they should relate to an office working environment.

15.8. Name four different types of selection criteria that might be used to select the most relevant rules for firing in a production system.

15.9. Describe a method in which rules could be grouped or organized in a knowledge base to reduce the amount of search required during the matching part of the inference cycle.

15.10. Using the knowledge base of Problem 15.7, simulate three match-select-execute cycles for a query which uses several rules and/or facts.

15.11. Explain the difference between forward and backward chaining and under what conditions each would be best to use for a given set of problems.

15.12. Under what conditions would it make sense to use both forward and backward chaining? Give an example where both are used.

15.13. Explain why you think associative networks were never very popular forms of knowledge representations in expert systems architectures.

15.14. Suppose you are diagnosing automobile engines using a system having a frame type of architecture similar to PIP. Show how a trigger condition might be satisfied for the distributor ignition system when it is learned that the spark at all spark plugs is weak.

15.15. Give the advantages of expert system architectures based on decision trees over those of production rules. What are the main disadvantages?

15.16. Two of the main problems in validating the knowledge contained in the knowledge bases of expert systems are related to completeness and consistency, that is, whether or not a system has an adequate breadth of knowledge to solve the class of problems it was intended to solve and whether or not the knowledge is consistent. Is it easier to check decision tree architectures or production rule systems for completeness and consistency? Give supporting information for your conclusions.

15.17. Give three examples of applications for which blackboard architectures are well suited.

15.18. Give three examples of applications for which the use of analogical architectures would be suitable in expert systems.

15.19. Consider a simple fully connected neural network containing three input nodes and a single output node. The inputs to the network are the eight possible binary patterns 000, 001, . . . , 111. Find weights w_i for which the network can differentiate between the inputs by producing three distinct outputs.

15.20. For the preceding problem, draw projection vectors on the unit circle for the eight different inputs using the weights determined there.

15.21. Explain how uncertainty is propagated through a chain of rules during a consultation with an expert system which is based on the MYCIN architecture.

15.22. Select a problem domain that requires some special expertise and consult with an

expert in the domain to learn how he or she solves typical problems. After collecting enough knowledge to solve a small subset of problems, create rules which could be used in a knowledge base to solve the problems. Test the use of the rules on a few problems which have been suggested by the expert and then get his or her confirmation.

15.23. Relate each of the heuristics given by Penny Nii in Section 15.5 to a real expert system solving problem.

15.24. Discuss how each of the features of expert system building tools given in Section 15.6 can affect the performance of the systems developed.

15.25. Obtain a copy of an expert system building tool such as Personal Consultant Plus and create an expert system to diagnose automobile engine problems. Consult with a mechanic to see if your completed system is reasonably good.

PART 5
Knowledge Acquisition

16
General Concepts in Knowledge Acquisition

The success of knowledge-based systems lies in the quality and extent of the knowledge available to the system. Acquiring and validating a large corpus of consistent, correlated knowledge is not a trivial problem. This has given the acquisition process an especially important role in the design and implementation of these systems. Consequently, effective acquisition methods have become one of the principal challenges for the AI research community.

16.1 INTRODUCTION

The goals in this branch of AI are the discovery and development of efficient, cost effective methods of acquisition. Some important progress has recently been made in this area with the development of sophisticated editors and some impressive machine learning programs. But much work still remains before truly general purpose acquisition is possible. In this chapter, we consider general concepts related to acquisition and learning. We begin with a taxonomy of learning based on definitions of behavioral learning types, assess the difficulty in collecting and assimilating large quantities of well correlated knowledge, describe a general model for learning, and examine different performance measures related to the learning process.

Definitions

Knowledge acquisition is the process of adding new knowledge to a knowledge base and refining or otherwise improving knowledge that was previously acquired. Acquisition is usually associated with some purpose such as expanding the capabilities of a system or improving its performance at some specified task. Therefore, we will think of acquisition as goal oriented creation and refinement of knowledge. We take a broad view of the definition here and include autonomous acquisition, contrary to many workers in the field who regard acquisition solely as the process of knowledge elicitation from experts.

Acquired knowledge may consist of facts, rules, concepts, procedures, heuristics, formulas, relationships, statistics, or other useful information. Sources of this knowledge may include one or more of the following.

Experts in the domain of interest
Textbooks
Technical papers
Databases
Reports
The environment

We will consider machine learning as a specialized form of acquisition. It is any method of autonomous knowledge creation or refinement through the use of computer programs.

Table 16.1 depicts several types of knowledge and possible representation structures which by now should be familiar. In building a knowledge base, it is

TABLE 16.1 TYPES OF KNOWLEDGE AND POSSIBLE STRUCTURES

Type of Knowledge	Examples of Structures
Facts	(snow color white)
Relations	(father_of john bill)
Rules	(if (temperature>200 degrees) (open relief_valve))
Concepts	(forall (x y) (if (and (male x) ((brother_of x) (or (father_of y) (mother_of y))) (uncle x y)]
Procedures, Plans, etc.	. . . . .

necessary to create or modify structures such as these for subsequent use by a performance component (like a theorem prover or an inference engine).

To be effective, the newly acquired knowledge should be integrated with existing knowledge in some meaningful way so that nontrivial inferences can be drawn from the resultant body of knowledge. The knowledge should, of course, be accurate, nonredundant, consistent (noncontradictory), and fairly complete in the sense that it is possible to reliably reason about many of the important conclusions for which the system was intended.

16.2 TYPES OF LEARNING

We all learn new knowledge through different methods, depending on the type of material to be learned, the amount of relevant knowledge we already possess, and the environment in which the learning takes place. It should not come as a surprise to learn that many of these same types of learning methods have been extensively studied in AI.

In what follows, it will be helpful to adopt a classification or taxonomy of learning types to serve as a guide in studying or comparing differences among them. One can develop learning taxonomies based on the type of knowledge representation used (predicate calculus, rules, frames), the type of knowledge learned (concepts, game playing, problem solving), or by the area of application (medical diagnosis, scheduling, prediction, and so on). The classification we will use, however, is intuitively more appealing and one which has become popular among machine learning researchers. The classification is independent of the knowledge domain and the representation scheme used. It is based on the type of inference strategy employed or the methods used in the learning process.

The five different learning methods under this taxonomy are

Memorization (rote learning)
Direct instruction (by being told)
Analogy
Induction
Deduction

Learning by memorization is the simplest form of learning. It requires the least amount of inference and is accomplished by simply copying the knowledge in the same form that it will be used directly into the knowledge base. We use this type of learning when we memorize multiplication tables, for example.

A slightly more complex form of learning is by direct instruction. This type of learning requires more inference than rote learning since the knowledge must be transformed into an operational form before being integrated into the knowledge base. We use this type of learning when a teacher presents a number of facts directly to us in a well organized manner.

The third type listed, analogical learning, is the process of learning a new concept or solution through the use of similar known concepts or solutions. We use this type of learning when solving problems on an exam where previously learned examples serve as a guide or when we learn to drive a truck using our knowledge of car driving. We make frequent use of analogical learning. This form of learning requires still more inferring than either of the previous forms, since difficult transformations must be made between the known and unknown situations.

The fourth type of learning is also one that is used frequently by humans. It is a powerful form of learning which, like analogical learning, also requires more inferring than the first two methods. This form of learning requires the use of inductive inference, a form of invalid but useful inference. We use inductive learning when we formulate a general concept after seeing a number of instances or examples of the concept. For example, we learn the concepts of color or sweet taste after experiencing the sensations associated with several examples of colored objects or sweet foods.

The final type of acquisition is deductive learning. It is accomplished through a sequence of deductive inference steps using known facts. From the known facts, new facts or relationships are logically derived. For example, we could learn deductively that Sue is the cousin of Bill, if we have knowledge of Sue and Bill's parents and rules for the cousin relationship. Deductive learning usually requires more inference than the other methods. The inference method used is, of course, a deductive type, which is a valid form of inference.

In addition to the above classification, we will sometimes refer to learning methods as either weak methods or knowledge-rich methods. Weak methods are general purpose methods in which little or no initial knowledge is available. These methods are more mechanical than the classical AI knowledge-rich methods. They often rely on a form of heuristic search in the learning process. Examples of some weak learning methods are given in the next chapter under the names of Learning Automata and Genetic Algorithms. We will be studying these and many of the more knowledge-rich forms of learning in more detail later, particularly various types of inductive learning.

16.3 KNOWLEDGE ACQUISITION IS DIFFICULT

One of the important lessons learned by AI researchers during the 1970s and early 1980s is that knowledge is not easily acquired and maintained. It is a difficult and time-consuming process. Yet expert and other knowledge-based systems require an abundant amount of well correlated knowledge to achieve a satisfactory level of intelligent performance. Typically, tens of person years are often required to build up a knowledge base to an acceptable level of performance. This was certainly true for the early expert systems such as MYCIN, DENDRAL, PROSPECTOR, and XCON. The acquisition effort encountered in building these systems provided the impetus for researchers to search for new efficient methods of acquisition. It helped to revitalize a new interest in general machine learning techniques.

Early expert systems initially had a knowledge base consisting of a few hundred rules. This is equivalent to less than 10^6 bits of knowledge. In contrast, the capacity of a mature human brain has been estimated at some 10^{15} bits of knowledge (Sagan, 1977). If we expect to build expert systems that are highly competent and possess knowledge in more than a single narrow domain, the amount of knowledge required for such knowledge bases will be somewhere between these two extremes, perhaps as much as 10^{10} bits.

If we were able to build such systems at even ten times the rate these early systems were built, it would still require on the order of 10^4 person years. This estimate is based on the assumption that the time required is directly proportional to the size of the knowledge base, a simplified assumption, since the complexity of the knowledge and the interdependencies grow more rapidly with the size of the knowledge base.

Clearly, this rate of acquisition is not acceptable. We must develop better acquisition and learning methods before we can implement such systems within a realistic time frame. Even with the progress made in the past few years through the development of special editors and related tools, more significant breakthroughs are needed before truly large knowledge bases can be assembled and maintained. Because of this, we expect the research interest in knowledge acquisition and machine learning to continue to grow at an accelerated rate for some years in the future.

It has been stated before that a system's performance is strongly dependent on the level and quality of its knowledge, and that "in knowledge lies power." If we accept this adage, we must also agree that the acquisition of knowledge is of paramount importance and, in fact, that "the real power lies in the ability to acquire new knowledge efficiently." To build a machine that can learn and continue to improve its performance has been a long time dream of mankind. The fulfillment of that dream now seems closer than ever before with the modest successes achieved by AI researchers over the past twenty years.

We will consider the complexity problem noted above again from a different point of view when we study the different learning paradigms.

16.4 GENERAL LEARNING MODEL

As noted earlier, learning can be accomplished using a number of different methods. For example, we can learn by memorizing facts, by being told, or by studying examples like problem solutions. Learning requires that new knowledge structures be created from some form of input stimulus. This new knowledge must then be assimilated into a knowledge base and be tested in some way for its utility. Testing means that the knowledge should be used in the performance of some task from which meaningful feedback can be obtained, where the feedback provides some measure of the accuracy and usefulness of the newly acquired knowledge.

A general learning model is depicted in Figure 16.1 where the environment has been included as part of the overall learner system. The environment may be regarded as either a form of nature which produces random stimuli or as a more

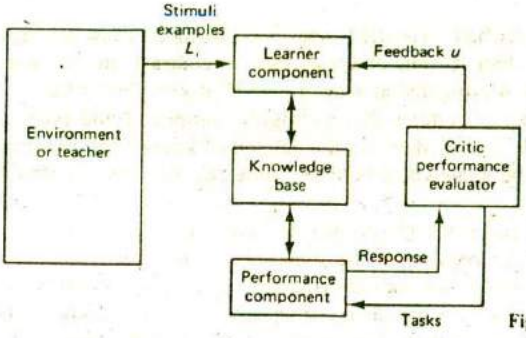

Figure 16.1 General learning model.

organized training source such as a teacher which provides carefully selected training examples for the learner component. The actual form of environment used will depend on the particular learning paradigm. In any case, some representation language must be assumed for communication between the environment and the learner. The language may be the same representation scheme as that used in the knowledge base (such as a form of predicate calculus). When they are chosen to be the same, we say the single representation trick is being used. This usually results in a simpler implementation since it is not necessary to transform between two or more different representations.

For some systems the environment may be a user working at a keyboard. Other systems will use program modules to simulate a particular environment. In even more realistic cases, the system will have real physical sensors which interface with some world environment.

Inputs to the learner component may be physical stimuli of some type or descriptive, symbolic training examples. The information conveyed to the learner component is used to create and modify knowledge structures in the knowledge base. This same knowledge is used by the performance component to carry out some tasks, such as solving a problem, playing a game, or classifying instances of some concept.

When given a task, the performance component produces a response describing its actions in performing the task. The critic module then evaluates this response relative to an optimal response.

Feedback, indicating whether or not the performance was acceptable, is then sent by the critic module to the learner component for its subsequent use in modifying the structures in the knowledge base. If proper learning was accomplished, the system's performance will have improved with the changes made to the knowledge base.

The cycle described above may be repeated a number of times until the performance of the system has reached some acceptable level, until a known learning goal has been reached, or until changes cease to occur in the knowledge base after some chosen number of training examples have been observed.

There are several important factors which influence a system's ability to learn in addition to the form of representation used. They include the types of training provided, the form and extent of any initial background knowledge, the type of feedback provided, and the learning algorithms used (Figure 16.2).

The type of training used in a system can have a strong effect on performance, much the same as it does for humans. Training may consist of randomly selected instances or examples that have been carefully selected and ordered for presentation. The instances may be positive examples of some concept or task being learned, they may be negative, or they may be a mixture of both positive and negative. The instances may be well focused using only relevant information, or they may contain a variety of facts and details including irrelevant data.

Many forms of learning can be characterized as a search through a space of possible hypotheses or solutions (Mitchell, 1982). To make learning more efficient, it is necessary to constrain this search process or reduce the search space. One method of achieving this is through the use of background knowledge which can be used to constrain the search space or exercise control operations which limit the search process. We will see several examples of this in the next three chapters.

Feedback is essential to the learner component since otherwise it would never know if the knowledge structures in the knowledge base were improving or if they were adequate for the performance of the given tasks. The feedback may be a simple yes or no type of evaluation, or it may contain more useful information describing why a particular action was good or bad. Also, the feedback may be completely reliable, providing an accurate assessment of the performance or it may contain noise; that is, the feedback may actually be incorrect some of the time. Intuitively, the feedback must be accurate more than 50% of the time; otherwise the system would never learn. If the feedback is always reliable and carries useful information, the learner should be able to build up a useful corpus of knowledge quickly. On the other hand, if the feedback is noisy or unreliable, the learning process may be very slow and the resultant knowledge incorrect.

Finally, the learning algorithms themselves determine to a large extent how successful a learning system will be. The algorithms control the search to find and build the knowledge structures. We then expect that the algorithms that extract much of the useful information from training examples and take advantage of any background knowledge outperform those that do not. In the following chapters we

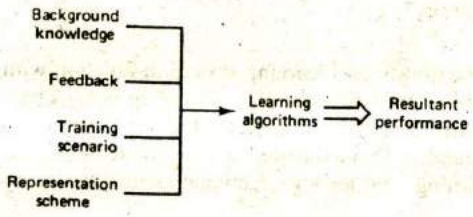

Figure 16.2 Factors affecting learning performance.

will see examples of systems which illustrate many of the above points regarding the effects of different factors on performance.

16.5 PERFORMANCE MEASURES

In the following four chapters, we will be investigating systems based on different learning paradigms and having different architectures. How can we evaluate the performance of a given system or compare the relative performance of two different systems? We could attempt to conduct something like a Turing test on a system. But would this tell us how general or robust the system is or how easy it is to implement? Clearly, such comparisons are possible only when standard performance measures are available. For example, it would be useful to establish the relative efficiencies or speed with which two systems learned a concept or how robust (noise tolerant) a system is under different training scenarios. Although little work has been done to date in this area, we will propose some informal definitions for performance measures in this section. These definitions will at least permit us to establish estimates of some relative performance characteristics among the different learning methods we will be considering.

Generality. One of the most important performance measures for learning methods is the generality or scope of the method. Generality is a measure of the ease with which the method can be adapted to different domains of application. A completely general algorithm is one which is a fixed or self adjusting configuration that can learn or adapt in any environment or application domain. At the other extreme are methods which function in a single domain only. Methods which have some degree of generality will function well in at least a few domains.

Efficiency. The efficiency of a method is a measure of the average time required to construct the target knowledge structures from some specified initial structures. Since this measure is often difficult to determine and is meaningless without some standard comparison time, a relative efficiency index can be used instead. For example, the relative efficiency of a method can be defined as the ratio of the time required for the given method to the time required for a purely random search to find the target structures.

Robustness. Robustness is the ability of a learning system to function with unreliable feedback and with a variety of training examples, including noisy ones. A robust system must be able to build tentative structures which are subject to modification or withdrawal if later found to be inconsistent with statistically sound structures. This is nonmonotonic learning, the analog of nonmonotonic reasoning discussed in Chapter 5.

Efficacy. The efficacy of a system is a measure of the overall power of the system. It is a combination of the factors generality, efficiency, and robustness. We say that system A is more efficacious than system B if system A is more efficient, robust, and general than B.

Ease of implementation. Ease of implementation relates to the complexity of the programs and data structures and the resources required to develop the given learning system. Lacking good complexity metrics, this measure will often be somewhat subjective.

Other performance terms that are specific to different paradigms will be introduced as needed.

16.6 SUMMARY

Knowledge acquisition is the purposeful addition or refinement of knowledge structures to a knowledge base for use by knowledge-based systems. Machine learning is the autonomous acquisition of knowledge through the use of computer programs. The acquired knowledge may consist of facts, concepts, rules, relations, plans, and procedures, and the source of the knowledge may be one or more of the following: data bases, textbooks, domain experts, reports, or the environment.

A useful taxonomy for learning is one that is based on the behavioral strategy employed in the learning process, strategies like rote (memorization), being told, analogy, induction, or deduction. Rote learning requires little inference, while the other methods require increasingly greater amounts.

An important lesson learned by expert systems researchers is that knowledge acquisition is difficult, often requiring tens of person years to assemble several hundred rules. This problem helped to revive active research in more autonomous forms of acquisition or machine learning.

Any model of learning should include components which support the basic learner component. These include a teacher or environment, a knowledge base, a performance component which uses the knowledge, and a critic or performance evaluation unit which provides feedback to the learner about the performance.

Factors which affect the performance of a learner system include (1) the representation scheme used, (2) the training scenario, (3) the type of feedback, (4) background knowledge, and (5) the learning algorithm.

Performance measures provide a means of comparing different performance characteristics of learning algorithms (systems). Some of the more important performance measures are generality, efficiency, robustness, efficacy, and ease of implementation. Lacking formal metrics for some measures will require that subjective estimates be used instead.

EXERCISES

16.1. Give a detailed definition of knowledge acquisition complete with an example.

16.2. Give an example for each of the following types of knowledge: (a) a fact, (b) a rule, (c) a concept, (d) a procedure, (e) a heuristic, (f) a relationship.

16.3. How is machine learning distinguished from general knowledge acquisition?

16.4. The taxonomy of learning methods given in this chapter is based on the type of behavior or inference strategy used. Give another taxonomy for learning methods and illustrate with some examples.

16.5. Describe the role of each component of a general learning model and why it is needed for the learning process.

16.6. Explain why a learning component should have scope.

16.7. What is the difference between efficiency and efficacy with regard to the performance of a learner system?

16.8. Review the knowledge acquisition section in Chapter 15 and explain why elicitation of knowledge from experts is so difficult.

16.9. Consult a good dictionary and describe the difference between induction and deduction. Give examples of both.

16.10. Explain why inductive learning should require more inference than learning by being told (instruction).

16.11. Try to determine whether inductive learning requires more inference than analogical learning. Give reasons for your conclusion. Which of the two types of learning, in general, would be more reliable in the sense that the knowledge learned is logically valid?

16.12. Give examples of each of the different types of learning as they relate to learning you have recently experienced.

16.13. Give an estimate of the number of bits required to store knowledge for 500 if . . . then rules, where each rule has five conjunctive terms or conditions in the antecedent. Can you estimate what fraction of your total knowledge is represented in some set of 500 rules?

16.14. Try to give a quantitative measure for knowledge. Explain why your measure is or is not reasonable. Give examples to support your arguments.

16.15. Explain why robustness in a learner is important in real world environments.

17

Early Work in Machine Learning

17.1 INTRODUCTION

Attempts to develop autonomous learning systems began in the 1950s while cybernetics was still an active area of research. These early designs were self-adapting systems which modified their own structures in an attempt to produce an optimal response to some input stimuli. Although several different approaches were pursued during this period, we will consider only four of the more representative designs. One approach was believed to be an approximate model of a small network of neurons.

A second approach was initially based on a form of rote learning. It was later modified to learn by adaptive parameter adjustment. The third approach used self-adapting stochastic automata models, while the fourth approach was modeled after survival of the fittest through population genetics.

In the remainder of this chapter, we will examine examples of the four methods and reserve our descriptions of more classical AI learning approaches for Chapters 18, 19, and 20. The systems we use as examples here are famous ones that have received much attention in the literature. They are Rosenblatt's perceptrons, Samuel's checkers playing system, Learning Automata, and Genetic Algorithms.

17.2 PERCEPTRONS

Perceptrons are pattern recognition or classification devices that are crude approximations of neural networks. They make decisions about patterns by summing up evidence obtained from many small sources. They can be taught to recognize one or more classes of objects through the use of stimuli in the form of labeled training examples.

A simplified perceptron system is illustrated in Figure 17.1. The inputs to the system are through an array of sensors such as a rectangular grid of light sensitive pixels. These sensors are randomly connected in groups to associative threshold units (ATU) where the sensor outputs are combined and added together. If the combined outputs to an ATU exceed some fixed threshold, the ATU unit executes and produces a binary output.

The outputs from the ATU are each multiplied by adjustable parameters or weights w_i $(i = 1, 2, \ldots, k)$ and the results added together in a terminal comparator unit. If the input to the comparator exceeds a given threshold level T, the perceptron produces a positive response of 1 (yes) corresponding to a sample classification of class-1. Otherwise, the output is 0 (no) corresponding to an object classification of non-class-1

All components of the system are fixed except the weights w_i which are adjusted through a punishment-reward process described below. This learning process contin-

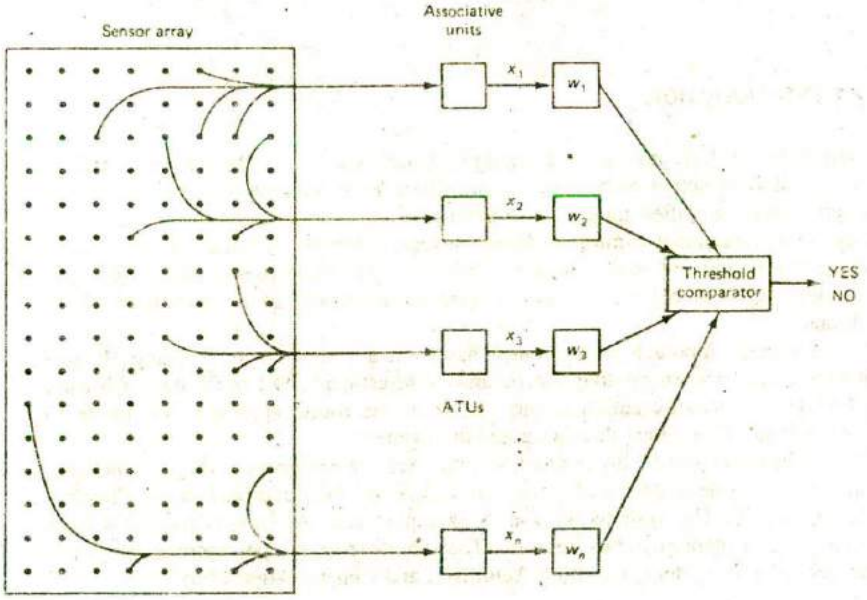

Figure 17.1 A simple perceptron.

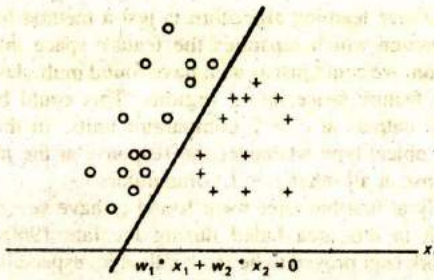

Figure 17.2 Geometrical illustration of separable space.

ues until optimal values of w_i are found at which time the system will have learned the proper classification of objects for the two different classes.

The light sensors produce an output voltage that is proportional to the light intensity striking them. This output is a measure of what is known in pattern recognition as the object representation space parameters. The outputs from the ATUs which combine several of the sensor outputs (x_i) are known as feature value measurements. These feature values are each multiplied by the weights w_i and the results summed in the comparator to give the vector product $\mathbf{r} = \mathbf{w} * \mathbf{x} = \Sigma_i\, w_i x_i$. When enough of the feature values are present and the weight vector is near optimal, the threshold will be exceeded and a positive classification will result.

Finding the optimal weight vector value w is equivalent to finding a separating hyperplane in k-dimensional space. If there is some linear function of the x_i for which objects in class-1 produce an output greater than T and non-class-1 objects produce an output less than T, the space of objects is said to be linearly separable (see Chapter 13 for example). Spaces which are linearly separable can be partitioned into two or more disjointed regions which divide objects based on their feature vector values as illustrated in Figure 17.2. It has been shown (Minsky and Papert, 1969) that an optimal w can always be found with a finite number of training examples if the space is linearly separable.

One of the simplest algorithms for finding an optimum w (w*) is based on the following perceptron learning algorithm. Given training objects from two distinct classes, class-1 and class-2

1. Choose an arbitrary initial value for w
2. After the m^{th} training step set

$$\mathbf{w}_{m+1} = \mathbf{w}_m + d * \mathbf{x}_m$$

where

$d = +1$ if $r = \mathbf{w} * \mathbf{x} < 0$ and $\mathbf{x}_m$ is type class-1

$d = -1$ if $r > 0$ and $\mathbf{x}_m$ is type class-2

Otherwise set $\mathbf{w}_{m+1} = \mathbf{w}_m$ $(d = 0)$

3. When $\mathbf{w}_m = \mathbf{w}_{m+j}$ (for all $j >= 1$) stop, the optimum w* has been found.

It should be recognized that the above learning algorithm is just a method for finding a linear two-class decision function which separates the feature space into two regions. For a generalized perceptron, we could just as well have found multiclass decision functions which separate the feature space into c regions. This could be done by terminating each of the ATU outputs at $c > 2$ comparator units. In this case, class j would be selected as the object type whenever the response at the jth comparator was greater than the response at all other $j - 1$ comparators.

Perceptrons were studied intensely at first but later were found to have severe limitations. Therefore, active research in this area faded during the late 1960s. However, the findings related to this work later proved to be most valuable, especially in the area of pattern recognition. More recently, there has been renewed interest in similar architectures which attempt to model neural networks (Chapter 15). This is partly due to a better understanding of the brain and significant advances realized in network dynamics as well as in hardware over the past ten years. These advances have made it possible to more closely model large networks of neurons.

17.3 CHECKERS PLAYING EXAMPLE

During the 1950s and 1960s Samuel (1959, 1967) developed a program which could learn to play checkers at a master's level. This system remembered thousands of board states and their estimated values. They provided the means to determine the best move to make at any point in the game.

Samuel's system learns while playing the game of checkers, either with a human opponent or with a copy of itself. At each state of the game, the program checks to see if it has remembered a best-move value for that state. If not, the program explores ahead three moves (it determines all of its possible moves; for each of these, it finds all of its opponent's moves; and for each of those, it determines all of its next possible moves). The program then computes an advantage or win-value estimate of all the ending board states. These values determine the best move for the system from the current state. The current board state and its corresponding value are stored using an indexed address scheme for subsequent recall.

The best move for each state is the move value of the largest of the minimums, based on the theory of two-person zero-sum games. This move will always be the best choice (for the next three moves) against an intelligent adversary.

As an example of the look-ahead process, a simple two move sequence is illustrated in Figure 17.3 in the form of a tree. At board state K, the program looks ahead two moves and computes the value of each possible resultant board state. It then works backward by first finding the minimum board values at state $K + 2$ in each group of moves made from state $K + 1$ (minimums = 4, 3, and 2).

These minimums correspond to the moves the opponent would make from each position when at state $K + 1$. The program then chooses the maximum of these minimums as the best (minimax) move it can make from the present board

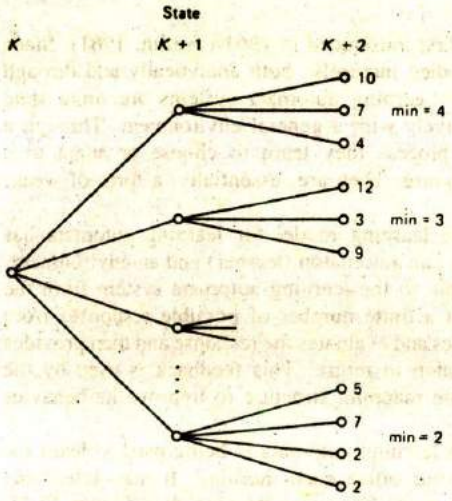

Figure 17.3 A two-move look-ahead sequence.

state K (maximum = 4). By looking ahead three moves, the system can be assured it can do no worse than this minimax value. The board state and the corresponding minimax value for a three-move-ahead sequence are stored in Samuel's system. These values are then available for subsequent use when the same state is encountered during a new game.

The look ahead search process could be extended beyond three moves; however, the combinatorial explosion that results makes this infeasible. But, when many board states have been learned, it is likely that any given state will already have look-ahead values for three moves, and some of those moves will in turn have look-ahead values stored. Consequently, as more and more values are stored, look-ahead values for six, nine, or even more moves may be prerecorded for rapid use. Thus, when the system has played many games and recorded thousands of moves, its ability to look ahead many moves and to show improved performance is greatly increased.

The value of a board state is estimated by computing a linear function similar to the perceptron linear decision function. In this case, however, Samuel selected some 16 board features from a larger set of feature parameters. The features were typically checkers concepts such as piece advantage, the number of kings and single piece units, and the location of pieces. In the original system, the weighting parameters were fixed. In subsequent experiments, however, the parameters were adjusted as part of the learning process much like the weighting parameters were in the perceptrons.

17.4 LEARNING AUTOMATA

The theory of learning automata was first introduced in 1961 (Tsetlin, 1961). Since that time these systems have been studied intensely, both analytically and through simulations (Lakshmivarahan, 1981). Learning automata systems are finite state adaptive systems which interact iteratively with a general environment. Through a probabilistic trial-and-error response process they learn to choose or adapt to a behavior which produces the best response. They are, essentially, a form of weak, inductive learners.

In Figure 17.4, we see that the learning model for learning automata has been simplified to just two components, an automaton (learner) and an environment. The learning cycle begins with an input to the learning automata system from the environment. This input elicits one of a finite number of possible responses from the automaton. The environment receives and evaluates the response and then provides some form of feedback to the automaton in return. This feedback is used by the automaton to alter its stimulus-response mapping structure to improve its behavior in a more favorable way.

As a simple example, suppose a learning automata is being used to learn the best temperature control setting for your office each morning. It may select any one of ten temperature range settings at the beginning of each day (Figure 17.5). Without any prior knowledge of your temperature preferences, the automaton randomly selects a first setting using the probability vector corresponding to the temperature settings.

Since the probability values are uniformly distributed, any one of the settings will be selected with equal likelihood. After the selected temperature has stabilized, the environment may respond with a simple good-bad feedback response. If the response is good, the automata will modify its probability vector by rewarding the probability corresponding to the good setting with a positive increment and reducing all other probabilities proportionately to maintain the sum equal to 1. If the response is bad, the automaton will penalize the selected setting by reducing the probability corresponding to the bad setting and increasing all other values proportionately. This process is repeated each day until the good selections have high probability values and all bad choices have values near zero. Thereafter, the system will always choose the good settings. If, at some point, in the future your temperature preferences change, the automaton can easily readapt.

Learning automata have been generalized and studied in various ways. One

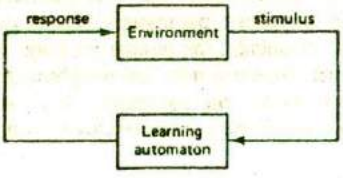

Figure 17.4 Learning automaton model.

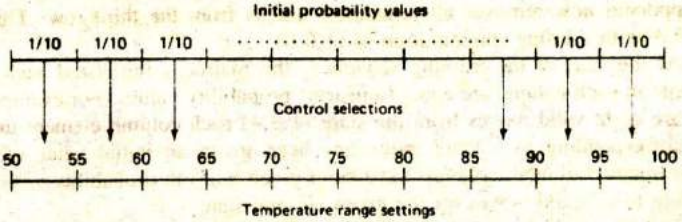

Figure 17.5 Temperature control model.

such generalization has been given the special name of collective learning automata (CLA). CLAs are standard learning automata systems except that feedback is not provided to the automaton after each response. In this case, several collective stimulus-response actions occur before feedback is passed to the automaton. It has been argued (Bock, 1976) that this type of learning more closely resembles that of human beings in that we usually perform a number or group of primitive actions before receiving feedback on the performance of such actions, such as solving a complete problem on a test or parking a car. We illustrate the operation of CLAs with an example of learning to play the game of Nim in an optimal way.

Nim is a two-person zero-sum game in which the players alternate in removing tokens from an array which initially has nine tokens. The tokens are arranged into three rows with one token in the first row, three in the second row, and five in the third row (Figure 17.6).

The first player must remove at least one token but not more than all the tokens in any single row. Tokens can only be removed from a single row during each player's move. The second player responds by removing one or more tokens remaining in any row. Players alternate in this way until all tokens have been removed; the loser is the player forced to remove the last token.

We will use the triple (n_1,n_2,n_3) to represent the states of the game at a given time where n_1, n_2, and n_3 are the numbers of tokens in rows 1, 2, and 3, respectively. We will also use a matrix to determine the moves made by the CLA for any given state. The matrix of Figure 17.7 has heading columns which correspond to the state of the game when it is the CLA's turn to move, and row headings which correspond to the new game state after the CLA has completed a move. Fractional entries in the matrix are transition probabilities used by the CLA to execute each of its moves. Asterisks in the matrix represent invalid moves.

Beginning with the initial state (1,3,5), suppose the CLA's opponent removes two tokens from the third row resulting in the new state (1,3,3). If the CLA then

Figure 17.6 Nim initial configuration.

removes all three tokens from the second row, the resultant state is (1,0,3). Suppose the opponent now removes all remaining tokens from the third row. This leaves the CLA with a losing configuration of (1,0,0).

At the start of the learning sequence, the matrix is initialized such that the elements in each column are equal (uniform) probability values. For example, since there are eight valid moves from the state (1,3,4) each column element under this state corresponding to a valid move has been given an initial value of $\frac{1}{8}$. In a similar manner all other columns have been given uniform probability values corresponding to all valid moves for the given column state.

The CLA selects moves probabilistically using the probability values in each column. So, for example, if the CLA had the first move, any row intersecting with the first column not containing an asterisk would be chosen with probability $\frac{1}{8}$. This choice then determines the new game state from which the opponent must select a move. The opponent might have a similar matrix to record game states and choose moves. A complete game is played before the CLA is given any feedback, at which time it is informed whether or not its responses were good or bad. This is the collective feature of the CLA.

If the CLA wins a game, all moves made by the CLA during that game are rewarded by increasing the probability value in each column corresponding to the winning move. All nonwinning probabilities in those columns are reduced equally to keep the sum in each column equal to 1. If the CLA loses a game, the moves leading to that loss are penalized by reducing the probability values corresponding to each losing move. All other probabilities in the columns having a losing move are increased equally to keep the column totals equal to 1.

After a number of games have been played by the CLA, the matrix elements

Current state

	135	134	133	132	...	125	...
135	•	•	•	•	...	•	
134	1/9	•	•	•	...	•	...
133	1/9	1/8	•	•	...	•	
132	1/9	1/8	1/7	•	...	•	...
⋮	⋮	⋮	⋮	⋮			
124	•	1/8	•	•	...	1/8	...
⋮	⋮	⋮					

Figure 17.7 CLA internal representation of game states.

which correspond to repeated wins will increase toward one, while all other elements in the column will decrease toward zero. Consequently, the CLA will choose the winning moves more frequently and thereby improve its performance.

Simulated games between a CLA and various types of opponents have been performed and the results plotted (Bock, 1985). It was shown, for example, that two CLAs playing against each other required about 300 games before each learned to play optimally. Note, however, that convergence to optimality can be accomplished with fewer games if the opponent always plays optimally (or poorly), since, in such a case, the CLA will repeatedly lose (win) and quickly reduce (increase) the losing (winning) move elements to zero (one). It is also possible to speed up the learning process through the use of other techniques such as learned heuristics.

Learning systems based on the learning automaton or CLA paradigm are fairly general for applications in which a suitable state representation scheme can be found. They are also quite robust learners. In fact, it has been shown that an LA will converge to an optimal distribution under fairly general conditions if the feedback is accurate with probability greater than 0.5 (Narendra and Thathachar, 1974). Of course, the rate of convergence is strongly dependent on the reliability of the feedback.

Learning automata are not very efficient learners as was noted in the game playing example above. They are, however, relatively easy to implement, provided the number of states is not too large. When the number of states becomes large, the amount of storage and the computation required to update the transition matrix becomes excessive.

Potential applications for learning automata include adaptive telephone routing and control. Such applications have been studied using simulation programs (Narendra et al., 1977). Although they have been given favorable recommendations, few if any actual systems have been implemented, however.

17.5 GENETIC ALGORITHMS

Genetic algorithm learning methods are based on models of natural adaptation and evolution. These learning systems improve their performance through processes which model population genetics and survival of the fittest. They have been studied since the early 1960s (Holland, 1962, 1975).

In the field of genetics, a population is subjected to an environment which places demands on the members. The members which adapt well are selected for mating and reproduction. The offspring of these better performers inherit genetic traits from both their parents. Members of this second generation of offspring which also adapt well are then selected for mating and reproduction and the evolutionary cycle continues. Poor performers die off without leaving offspring. Good performers produce good offspring and they, in turn, perform well. After some number of generations, the resultant population will have adapted optimally or at least very well to the environment.

Genetic algorithm systems start with a fixed size population of data structures

which are used to perform some given tasks. After requiring the structures to execute the specified tasks some number of times, the structures are rated on their performance, and a new generation of data structures is then created. The new generation is created by mating the higher performing structures to produce offspring. These offspring and their parents are then retained for the next generation while the poorer performing structures are discarded. The basic cycle is illustrated in Figure 17.8.

Mutations are also performed on the best performing structures to insure that the full space of possible structures is reachable. This process is repeated for a number of generations until the resultant population consists of only the highest performing structures.

Data structures which make up the population can represent rules or any other suitable types of knowledge structure. To illustrate the genetic aspects of the problem, assume for simplicity that the population of structures are fixed-length binary strings such as the eight bit string 11010001. An initial population of these eight-bit strings would be generated randomly or with the use of heuristics at time zero. These strings, which might be simple condition and action rules, would then be assigned some tasks to perform (like predicting the weather based on certain physical and geographic conditions or diagnosing a fault in a piece of equipment).

After multiple attempts at executing the tasks, each of the participating structures would be rated and tagged with a utility value u commensurate with its performance. The next population would then be generated using the higher performing structures as parents and the process would be repeated with the newly produced generation. After many generations the remaining population structures should perform the desired tasks well.

Mating between two strings is accomplished with the *crossover* operation which randomly selects a bit position in the eight-bit string and concatenates the head of

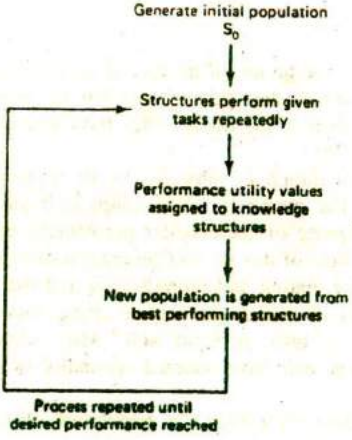

Figure 17.8 Genetic algorithm.

one parent to the tail of the second parent to produce the offspring. Suppose the two parents are designated as xxxxxxxx and yyyyyyyy respectively, and suppose the third bit position has been selected as the crossover point (at the position of the colon in the structure xxx:xxxxx). After the crossover operation is applied, two offspring are then generated, namely xxxyyyyy and yyyxxxxx. Such offspring and their parents are then used to make up the next generation of structures.

A second genetic operation often used is called *inversion*. Inversion is a transformation applied to a single string. A bit position is selected at random, and when applied to a structure, the inversion operation concatenates the tail of the string to the head of the same string. Thus, if the sixth position were selected ($x_1x_2x_3x_4x_5x_6:x_7x_8$), the inverted string would be $x_7x_8x_1x_2x_3x_4x_5x_6$.

A third operator, *mutation*, is used to insure that all locations of the rule space are reachable, that every potential rule in the rule space is available for evaluation. This insures that the selection process does not get caught in a local minimum. For example, it may happen that use of the crossover and inversion operators will only produce a set of structures that are better than all local neighbors but not optimal in a global sense. This can happen since crossover and inversion may not be able to produce some undiscovered structures. The mutation operator can overcome this by simply selecting any bit position in a string at random and changing it. This operator is typically used only infrequently to prevent random wandering in the search space.

The genetic paradigm is best understood through an example. To illustrate similarities between the learning automaton paradigm and the genetic paradigm we use the same learning task of the previous section, namely learning to play the game of nim optimally. We use a slightly different representation scheme here since we want a population of structures that are easily transformed. To do this, we let each member of the population consist of a pair of triplets augmented with a utility value u, $((n_1,n_2,n_3)\ (m_1,m_2,m_3)u)$, where the first pair is the game state presented to the genetic algorithm system prior to its move, and the second triple is the state after the move. The u values represent the worth or current utility of the structure at any given time.

Before the game begins, the genetic system randomly generates an initial population of K triple-pair members. The population size K is one of the important parameters that must be selected. Here, we simply assume it is about 25 or 30, which should be more than the number of moves needed for any optimal play. All members are assigned an initial utility value of 0. The learning process then proceeds as follows.

1. The environment presents a valid triple to the genetic system.
2. The genetic system searches for all population triple pairs which have a first triple that matches the input triple. From those that match, the first one found having the highest utility value u is selected and the second triple is returned as the new game state. If no match is found, the genetic system randomly generates a triple which represents a valid move, returns this as the new state,

and stores the triple pair, the input, and newly generated triple as an addition to the population.

3. The above two steps are repeated until the game is terminated, in which case the genetic system is informed whether a win or loss occurred. If the system wins, each of the participating member moves has its utility value increased. If the system loses, each participating member has its utility value decreased.
4. The above steps are repeated until a fixed number of games have been played. At this time a new generation is created.

The new generation is created from the old population by first selecting a fraction (say one half) of the members having the highest utility values. From these, offspring are obtained by application of appropriate genetic operators.

The three operators, crossover, inversion, and mutation, randomly modify the parent moves to give new offspring move sequences. (The best choice of genetic operators to apply in this example is left as an exercise). Each offspring inherits a utility value from one of the parents. Population members having low utility values are discarded to keep the population size fixed.

This whole process is repeated until the genetic system has learned all the optimal moves. This can be determined when the parent population ceases to change or when the genetic system repeatedly wins.

The similarity between the learning automaton and genetic paradigms should be apparent from this example. Both rely on random move sequences, and the better moves are rewarded while the poorer ones are penalized.

17.6 INTELLIGENT EDITORS

In the previous chapter, we considered some of the difficulties involved in acquiring and assembling a large corpus of well-correlated knowledge. Expert systems sometimes require hundreds or even thousands of rules to reach acceptable levels of performance. To help alleviate this problem, the development of special editors such as TEIRESIAS (Davis and Lenat, 1982) were initiated during the mid 1970s. Since that time a number of commercial editors have been developed. These intelligent editors have made it possible to build expert systems without strong reliance on knowledge engineers.[1]

An intelligent editor acts as an interface between a domain expert and an expert system. They permit a domain expert to interact directly with the system without the need for an intermediary to code the knowledge (Figure 17.9). The expert carries on a dialog with the editor in a restricted subset of English which includes a domain-specific vocabulary. The editor has direct access to the knowledge in the expert system and knows the structure of that knowledge. Through the editor,

[1] For additional details related to the overall process of eliciting, coding, organizing, and refining knowledge from domain experts, see Chapter 15 which examines expert system architectures and building tools.

Figure 17.9 Acquisition using an intelligent editor.

an expert can create, modify, and delete rules without a knowledge of the internal structure of the rules.

The editor assists the expert in building and refining a knowledge base by recalling rules related to some specific topic, and reviewing and modifying the rules, if necessary, to better fit the expert's meaning and intent. Through the editor, the expert can query the expert system for conclusions when given certain facts. If the expert is unhappy with the results, a trace can be obtained of the steps followed in the inference process. When faulty or deficit knowledge is found, the problem can then be corrected.

Some editors have the ability to suggest reasonable alternatives and to prompt the expert for clarifications when required. Some editors also have the ability to make validity checks on newly entered knowledge and detect when inconsistencies occur. More recently, a few commercial editors have incorporated features which permit rules to be induced directly from examples of problem solutions. These editors have greatly simplified the acquisition process, but they still require much effort on the part of domain experts.

17.7 SUMMARY

We have examined examples of early work done in machine learning including perceptrons which learn through parameter adjustment, by looking at Samuel's checkers playing system which learns through a form of rote learning as well as parameter adjustment. We then looked at learning automata which uses a reward and punishment process by modifying their state probability transformation mapping structures until optimal performance has been achieved. Genetic algorithm systems learn through a form of mutation and genetic inheritance. Higher performing knowledge structures are mated and give birth to offspring which possess many of their parents' traits. Generations of structures are created until an acceptable level of performance has been reached. Finally, we briefly discussed semiautonomous learning systems, the intelligent editors. These systems permit a domain expert to interact directly in building and refining a knowledge base without strong support from a knowledge engineer.

EXERCISES

17.1. Given a simple perceptron with a 3-x-3 input sensor array, compute six learning cycles to show how the weights w_i change during the learning process. Assign random weights to the initial w_i values.

17.2. For the game of checkers with an assumed average number of 50 possible moves per board position, determine the difference in the total number of moves for a four move look-ahead as compared to a three move look-ahead system.

17.3. Design a learning automaton that selects TV channels based on day of week and time of day (three evening hours only) for some family you are familiar with.

17.4. Write a computer program to simulate the learning automaton of the previous problem. Determine the number of training examples required for the system to converge to the optimal values.

17.5. Describe how a learning automaton could be developed to learn how to play the game of tic-tac-toe optimally. Is this a CLA or a simple learning automaton system?

17.6. Describe the similarities and differences between learning automata and genetic algorithms. Which learner would be best at finding optimal solutions to nonlinear functions? Give reasons to support your answer.

17.7. Explain the difference of the genetic operators inversion, crossover, and mutation. Which operator do you think is most effective in finding the optimal population in the least time?

17.8. Explain why some editors can be distinguished as "intelligent."

17.9. Read an article on TEIRESIUS and make a list of all the intelligent functions it performs that differ from so called nonintelligent editors.

18

Learning by Induction

18.1 INTRODUCTION

Consider playing the following game. I will choose some concept which will remain fixed throughout the game. You will then be given clues to the concept in the form of simple descriptions. After each clue, you must attempt to guess the concept I have chosen. I will continue with the clues until you are sure you have made the right choice.

Clue 1. A diamond ring.

For your first guess did you choose the concept beautiful? If so you are wrong.

Clue 2. Dinner at Maxime's restaurant.

What do clues 1 and 2 have in common? Perhaps clue 3 will be enough to reveal the commonality you need to make a correct choice.

Clue 3. A Mercedes-Benz automobile.

You must have discovered the concept by now! It is an expensive or luxury item.

The above illustrates the process we use in inductive learning, namely, inductive inference, an invalid, but useful form of inference.

In this chapter we continue the study of machine learning, but in a more focused manner. Here, we study a single learning method, learning by inductive inference.

Inductive learning is the process of acquiring generalized knowledge from examples or instances of some class. This form of learning is accomplished through inductive inference, the process of reasoning from a part to a whole, from particular instances to generalizations, or from the individual to the universal. It is a powerful form of learning which we humans do almost effortlessly. Even though it is not a valid form of inference, it appears to work well much of the time. Because of its importance, we have devoted two complete chapters to the subject.

18.2 BASIC CONCEPTS

When we conclude that October weather is always pleasant in El Paso after having observed the weather there for a few seasons, or when we claim that all swans are white after seeing only a small number of white swans, or when we conclude that all Scots are tough negotiators after conducting business with only a few, we are learning by induction. Our conclusions may not always be valid, however. For example, there are also black Australian swans and some weather records show that October weather in El Paso was inclement. Even so, these conclusions or rules are useful. They are correct much or most of the time, and they allow us to adjust our behavior and formulate important decisions with little cognitive effort.

One can only marvel at our ability to formulate a general rule for a whole class of objects, finite or not, after having observed only a few examples. How is it we are able to make this large inductive leap and arrive at an accurate conclusion so easily? For example, how is it that a first time traveler to France will conclude that all French people speak French after having spoken only to one Frenchman named Henri? At the same time, our traveler would not incorrectly conclude that all Frenchmen are named Henri!

Examples like this emphasize the fact that inductive learning is much more than undirected search for general hypotheses. Indeed, it should be clear that inductive generalization and rule formulation are performed in some context and with a purpose. They are performed to satisfy some objectives and, therefore, are guided by related background or other world knowledge. If this were not so, our generalizations would be on shaky ground and our class descriptions might be no more than a complete listing of all the examples we had observed.

The inductive process can be described symbolically through the use of predicates P and Q. If we observe the repeated occurrence of events $P(a_1)$, $P(a_2)$, . . . , $P(a_k)$, we generalize by inductively concluding that $\forall x\ P(x)$, i.e. if (canary_1 color yellow), (canary_2 color yellow), . . . , (canary_k color yellow) then (forall x (if canary x)(x color yellow)). More generally, when we observe the implications

$$P(a_1) \rightarrow Q(b_1)$$
$$P(a_2) \rightarrow Q(b_2)$$
$$\vdots$$
$$P(a_k) \rightarrow Q(b_k)$$

we generalize by concluding $\forall xy\ P(x) \rightarrow Q(y)$.

In forming a generalized hypothesis about a complete (possibly infinite) class after observing only a fraction of the members, we are making an uncertain conclusion. Even so, we have all found it to be a most essential form of learning.

Our reliance on and proven success with inductive learning has motivated much research in this area. Numerous systems have been developed to investigate different forms of such learning referred to as learning by observation, learning by discovery, supervised learning, learning from examples, and unsupervised learning. We have already seen two examples of weak inductive learning paradigms in the previous chapter: learning automata and genetic algorithms. In this chapter we will see different kinds of examples which use the more traditional AI architecture.

In the remainder of this and the following chapter, we will study the inductive learning process and look at several traditional AI learning systems based on inductive learning paradigms. We begin in the next two sections with definitions of important terms and descriptions of some essential tools used in the learning systems which follow.

18.3 SOME DEFINITIONS

In this section we introduce some important terms and concepts related to inductive learning. Many of the terms we define will have significance in other areas as well.

Our model of learning is the model described in Section 16.4. As you may recall, our learner component is presented with training examples from a teacher or from the environment. The examples may be positive instances only or both positive and negative. They may be selected, well-organized examples or they may be presented in a haphazard manner and contain much irrelevant information. Finally, the examples may be correctly labeled as positive or negative instances of some concept, they may be unlabeled, or they may even contain erroneous labels. Whatever the scenario, we must be careful to describe it appropriately.

Given (1) the observations, (2) certain background domain knowledge, and (3) goals or other preference criteria, the task of the learner is to find an inductive assertion or target concept that implies all of the observed examples and is consistent

with both the background knowledge and goals. We formalize the above ideas with the following definitions (Hunt et al., 1966, and Rendell, 1985).

Definitions

Object. Any entity, physical or abstract, which can be described by a set of attribute (feature) values and attribute relations is an object. We will refer to an object either by its name o_i or by an appropriate representation such as the vector of attribute values $\mathbf{x} = (x_1, x_2, \ldots, x_n)$. Clearly, the x_i can be very primitive attributes or higher level, more abstract ones. The choice will depend on the use being made of the representations. For example, a car may be described with primitives as an entity made from steel, glass, rubber, and its other component materials or, at a higher level of abstraction, as an object used for the transportation of passengers at speeds ranging up to 250 miles per hour.

Class. Given some universe of objects U, a class is a subset of U. For example, given the universe of four-legged animals, one class is the subset horses.

Concept. This is a description of some class (set) or rule which partitions the universe of objects U into two sets, the set of objects that satsisfy the rule and those that do not. Thus, the concept of horse is a description or rule which asserts the set of all horses and excludes all nonhorses.

Hypothesis. A hypothesis H is an assertion about some objects in the universe. It is a candidate or tentative concept which partitions the universe of objects. One such hypothesis related to the concept of horse is the class of four-legged animals with a tail. This is a candidate (albeit incomplete) for the concept horse.

Target concept. The target is one concept which correctly classifies all objects in the universe.

Positive instances. These are example objects which belong to the target concept.

Negative instances. These are examples opposite to the target concept.

Consistent classification rule. This is a rule that is true for all positive instances and false for all negative instances.

Induction. Induction is the process of class formation. Here we are more interested in the formation of classes which are goal-oriented; therefore, we will define induction as purposeful class formation. To illustrate, from our earlier example, we concluded that all Scotsmen are tough negotiators. Forming this concept can be helpful when dealing with any Scot. The goal or purpose here is in the simplification of our decisions when found in such situations.

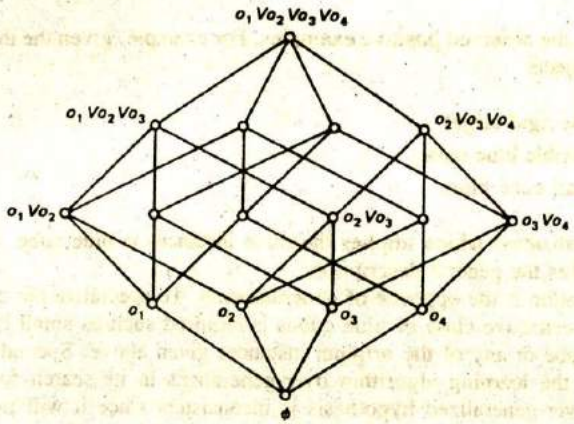

Figure 18.1 Lattice of object classes. Each node represents a disjunction of objects from a universe of four objects.

Selective induction. In this form of induction class descriptions are formed using only the attributes and relations which appear in the positive instances.

Constructive induction. This form of induction creates new descriptors not found in any of the instances.

Expedient induction. This is the application of efficient, efficacious inductive learning methods which have some scope, methods which span more than a single domain. The combined performance in efficiency, efficacy, and scope has been termed the inductive power of a system (Rendell, 1985).

In this chapter we are primarily interested in learners which exhibit expedient induction. Our reason for this concern will become more apparent when we examine the complexity involved in locating a target concept, even in a small universe. Consider, for example, the difficulty in locating a single concept class in a universe consisting of only four objects. Since the universe of all dichotomous sets (the concept C and U-C) containing n objects can be represented as a lattice structure having 2^n nodes, we see that this is an exponential search problem (Figure 18.1).

18.4 GENERALIZATION AND SPECIALIZATION

In this section we consider some techniques which are essential for the application of inductive learning algorithms. Concept learning requires that a guess or estimate of a larger class, the target concept, be made after having observed only some fraction of the objects belonging to that class. This is essentially a process of generalization, of formulating a description or a rule for a larger class but one which is still

consistent with the observed positive examples. For example, given the three positive instances of objects

(blue cube rigid large)
(small flexible blue cube)
(rigid small cube blue)

a proper generalization which implies the three instances is blue cube. Each of the instances satisfies the general description.

Specialization is the opposite of generalization. To specialize the concept blue cube, a more restrictive class of blue cubes is required such as small blue cube or flexible blue cube or any of the original instances given above. Specialization may be required if the learning algorithm over-generalizes in its search for the target concept. An over-generalized hypothesis is inconsistent since it will include some negative instances in addition to the positive ones.

There are many ways to form generalizations. We shall describe the most commonly used rules below. They will be sufficient to describe all of the learning paradigms which follow. In describing the rules, we distinguish between two basic types of generalization, comparable to the corresponding types of induction (Section 18.2), selective generalization and constructive generalization (Michalski, 1983). Selective generalization rules build descriptions using only the descriptors (attributes and relations) that appear in the instances, whereas constructive generalization rules do not. These concepts are described further below.

Generalization Rules

Since specialization rules are essentially the opposite of rules for generalization, to specialize a description, one could change variables to constants, add a conjunct or remove a disjunct from a description, and so forth. Of course, there are other means of specialization such as taking exceptions in descriptions (a fish is anything that swims in water but does not breathe air as do dolphins). Such methods will be introduced as needed.

These methods are useful tools for constructing knowledge structures. They give us methods with which to formulate and express inductive hypotheses. Unfortunately, they do not give us much guidance on how to select hypotheses efficiently. For this, we need methods which more directly limit the number of hypotheses which must be considered.

Selective Generalization

Changing Constants to Variables.

An example of this rule was given in Section 18.1. Given instances of a description or predicate $P(a_1)$, $P(a_2)$, . . . , $P(a_k)$ the constants a_i are changed to a variable which may be any value in the given domain, that is, $\forall x\ P(x)$.

Dropping Condition.

Dropping one or more conditions in a description has the effect of expanding or increasing the size of the set. For example, the set of all small red spheres is less general than the set of small spheres. Another way of stating this rule when the conjunctive description is given is that a generalization results when one or more of the conjuncts is dropped.

Adding an Alternative.

This is similar to the dropping condition rule. Adding a disjunctive term generalizes the resulting description by adding an alternative to the possible objects. For example, transforming red sphere to (red sphere) V (green pyramid) expands the class of red spheres to the class of red spheres or green pyramids. Note that the *internal disjunction* could also be used to generalize. An internal disjunction is one which appears inside the parentheses such as (red V green sphere).

Climbing a Generalization Tree.

When the classes of objects can be represented as a tree hierarchy as pictured in Figure 18.2, generalization is accomplished by simply climbing the tree to a node which is higher and, therefore, gives a more general description. For example, moving up the tree one level from elephant we obtain the more general class description of mammal. A greater generalization would be the class of all animals.

Closing an Interval.

When the domain of a descriptor is ordered ($d_1 < d_2 < \ldots < d_k$) and a few values lie in a small interval, a more restricted form of generalization than changing constants to variables can be performed by generalizing the values to a closed interval. Thus, if two or more instances with values $D = d_i$ and $D = d_j$

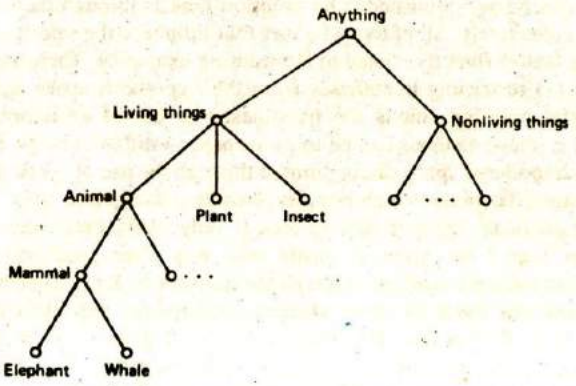

Figure 18.2 Generalization tree for the hierarchy of All Things.

where $d_i < d_j$ have been observed, the generalization $D = [d_i \ldots d_j]$ can be made; that is, D can be any value in the interval d_i to d_j.

Constructive Generalization

Generating Chain Properties.

If an order exists among a set of objects, they may be described by their ordinal position such as first, second, . . . , n^{th}. For example, suppose the relations for a four story building are given as

$$\text{above}(f_2,f_1) \;\&\; \text{above}(f_3,f_2) \;\&\; \text{above}(f_4,f_3)$$

then a constructive generalization is

$$\text{most\_above}(f_4) \;\&\; \text{least\_above}(f_1).$$

The most above, least above relations are created. They did not occur in the original descriptors.

Other forms of less frequently used generalization techniques are also available including combinations of the above. We will introduce such methods as we need them.

18.5 INDUCTIVE BIAS

Learning generalizations has been characterized as a search problem (Mitchell, 1982). We saw in Section 18.2 that learning a target concept is equivalent to finding a node in a lattice of 2^n nodes when there are n elementary objects. How can one expect to realize expedient induction when an exponential space must be searched? From our earlier exposure to search problems, we know that the naive answer to this question is simply to reduce the number of hypotheses which must be considered. But how can this be accomplished? Our solution here is through the use of *bias*.

Bias is, collectively, all of those factors that influence the selection of hypotheses, excluding factors directly related to the training examples. There are two general types of bias: (1) restricting hypotheses from the hypothesis space and (2) the use of a preferential ordering among the hypotheses or use of an informed selection scheme. Each of these methods can be implemented in different ways. For example, the size of the hypothesis space can be limited through the use of syntactic constraints in a representation language which permits attribute descriptions only. This will be the case with predicate calculus descriptions if only unary predicates are allowed, since relations cannot be expressed easily with one place predicates. Of course, such descriptions must be expressive enough to represent the knowledge being learned.

Representations based on more abstract descriptions will often limit the size of the space as well. Consider the visual scene of Figure 18.3. When used as a training example for concepts like on top of, the difference in the size of the hypothesis space between representations based on primitive pixel values and more abstract

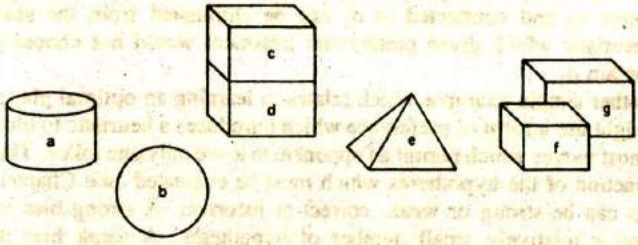

Figure 18.3 Blocks world scene.

descriptions based on a semantic net can be very large. For example, a representation using only two levels of gray (light and dark) in a 1024 by 1024 pixel array will have a hypothesis space in excess of 2^{10^6}. Compare this with the semantic net space which uses no more than 10 to 20 objects and a limited number of position relationships. Such a space would have no more than 10^4 or 10^5 object-position relationships. We see then that the difference in the size of the search space for these two representations can be immense.

Another simple example which limits the number of hypotheses is illustrated in Figure 18.4. The tree representation on the left contains more information and, therefore, will permit a larger number of object descriptions to be created than with the tree on the right. On the other hand, if one is only interested in learning general descriptions of geometrical objects without regard to details of size, the tree on the right will be a superior choice since the smaller tree will result in less search.

Methods based on the second general type of bias limit the search through preferential hypotheses selection. One way this can be achieved is through the use of heuristic evaluation functions. If it is known that a target concept should not contain some object or class of objects, all hypotheses which contain these objects can be eliminated from consideration. Referring again to Figure 18.1, if it is known

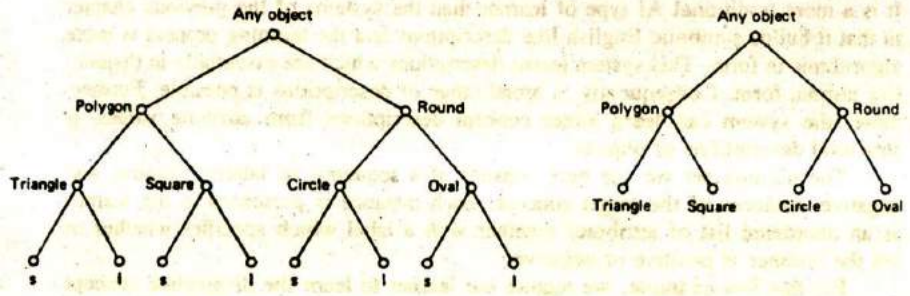

Figure 18.4 Tree representation for object descriptions (s = small, l = large).

that object o_3 (or the description of o_3) should not be included in the target set, all nodes above o_3 and connected to o_3 can be eliminated from the search. In this case, a heuristic which gives preferential treatment would not choose descriptions which contain o_3.

Another simple example which relates to learning an optimal play of the game of Nim might use a form of preference which introduces a heuristic to block consideration of most moves which permit an opponent to leave only one token. This eliminates a large fraction of the hypotheses which must be evaluated (see Chapter 17).

Bias can be strong or weak, correct or incorrect. A strong bias is one which focuses on a relatively small number of hypotheses. A weak bias does not. A correct bias is one which allows the learner to consider the target concept, whereas an incorrect bias does not. Obviously, a learner's task is simplified when the bias is both strong and correct (Utgoff, 1986). Bias can also be implemented in a program as either static or dynamic. When dynamic bias is employed, it is shifted automatically by the program to improve the learner's performance. We will see different forms of bias used in subsequent sections.

18.6 EXAMPLE OF AN INDUCTIVE LEARNER

Many learning programs have been implemented which construct descriptions composed of conjunctive features only. Few have been implemented to learn disjunctive descriptions as well. This is because conjunctive learning algorithms are easier to implement. Of course, a simple implementation for a disjunctive concept learner would be one which simply forms the disjunction of all positive training instances as the target concept. Obviously, this would produce an awkward description if there were many positive instances.

There are many concepts which simply cannot be described well in conjunctive terms only. One of the best examples is the concept of uncle since an uncle can be *either* the brother of the father *or* the brother of the mother of a child. To state it any other way is cumbersome.

The system we describe below was first implemented at M.I.T (Iba, 1979). It is a more traditional AI type of learner than the systems of the previous chapter in that it builds symbolic English-like descriptions and the learning process is more algorithmic in form. This system learns descriptions which are essentially in disjunctive normal form. Consequently, a broad range of descriptions is possible. Furthermore, the system can learn either concept descriptions from attribute values or structural descriptions of objects.

The training set we use here consists of a sequence of labeled positive and negative instances of the target concept. Each instance is presented to the learner as an unordered list of attributes together with a label which specifies whether or not the instance is positive or negative.

For this first example, we require our learner to learn the disjunctive concept "something that is either a tall flower or a yellow object." One such instance of

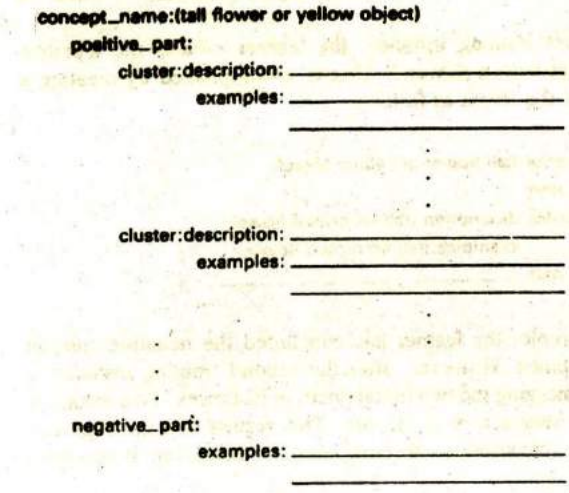

Figure 18.5 Frame-like knowledge structure.

this concept is represented as (short skinny yellow flower +), whereas a negative instance is (brown fat tall weed −). Given a number of positive and negative training instances such as these, the learner builds frame-like structures with groups of slots we will call clusters as depicted in Figure 18.5.

The target concept is given in the concept name. The actual description is then built up as a group of slots labeled as clusters. All training examples, both positive and negative, are retained in the example slots for use or reuse in building up the descriptions. An example will illustrate the basic algorithm.

Garden World Example

Each cluster in the frame of Figure 18.5 is treated as a disjunctive term, and descriptions within each cluster are treated as conjuncts. A complete learning cycle will clarify the way in which the clusters and frames (concepts) are created. We will use the following training examples from a garden world to teach our learner the concept "tall flower or yellow object."

(tall fat brown flower +)
(green tall skinny flower +)
(skinny short yellow weed +)
(tall fat brown weed −)
(fat yellow flower tall +)

After accepting the first training instance, the learner creates the tentative concept hypothesis "a tall fat brown flower." This is accomplished by creating a cluster in the positive part of the frame as follows:

```
concept_name:(tall flower or yellow object)
  positive_part:
        cluster:description:(tall fat brown flower)
              :examples:(tall fat brown flower)
  negative_part: _____________________________
```

With only a single example, the learner has concluded the tentative concept must be the same as the instance. However, after the second training instance, a new hypothesis is created by merging the two initial positive instances. Two instances are merged by taking the set intersection of the two. This results in a more general description, but one which is consistent with both positive examples. It produces the following structure.

```
concept_name:(tall flower or yellow object)
  positive_part:
        cluster:description:(tall, flower)
              :examples:(tall fat brown flower)
                        (green tall skinny flower)
  negative_part:
        examples: _____________________________
```

The next training example is also a positive one. Therefore, the set intersection of this example and the current description is formed when the learner is presented with this example. The resultant intersection and new hypothesis is an over generalization, namely, the null set, which stands for anything.

```
concept_name:(tall flower or yellow object)
  positive_part:
        cluster:description:()
              :examples:(tall fat brown flower)
                        (green tall skinny flower)
                        (skinny short yellow weed)
  negative_part:
              :examples: _____________________________
```

The fourth instance is a negative one. This instance is inconsistent with the current hypothesis which includes anything. Consequently, the learner must revise its hypothesis to exclude this last instance.

It does this by splitting the first cluster into two new clusters which are then both compatible with the negative instance. Each new cluster corresponds to a disjunctive term in this description.

To build the new clusters, the learner uses the three remembered examples from the first cluster. It merges the examples in such a way that each merge produces new consistent clusters. After merging we get the following revised frame.

```
concept_name:(tall flower or yellow object)
 positive_part:
      cluster:description:(tall flower)
          :examples:(tall fat brown flower)
                    (green tall skinny flower)
      cluster:description:(skinny short yellow weed)
          :examples:(skinny short yellow weed)
 negative_part:
          :examples:(tall fat brown weed)
```

The reader should verify that this new description has now excluded the negative instance.

The next training example is all that is required to arrive at the target concept. To complete the description, the learner attempts to combine the new positive instance with each cluster by merging as before, but only if the resultant merge is compatible with all negative instances (one in this case). If the new instance cannot be merged with any existing cluster without creating an inconsistency, a new cluster is created.

Merging the new instance with the first cluster results in the same cluster. Merging it with the second cluster produces a new, more general cluster description of yellow. The final frame obtained is as follows.

```
concept_name:(tall flower or yellow object)
 positive_part:
      cluster:description:(tall flower)
          :examples:(tall fat brown flower)
                    (green tall skinny flower)
                    (fat yellow flower tall)

      cluster:description:(yellow)
          :examples:(skinny short yellow weed)
                    (fat yellow flower tall)
 negative_part:
          :examples:(tall fat brown weed)
```

The completed concept now matches the target concept "tall flower or yellow object."

The above example illustrates the basic cycle but omits some important factors. First, the order in which the training instances are presented to the learner is important. Different orders, in general, will result in different descriptions and may require different numbers of training instances to arrive at the target concept.

Second, when splitting and rebuilding clusters after encountering a negative example, it is possible to build clusters which are not concise or maximal in the sense that some of the clusters could be merged without becoming inconsistent. Therefore, after rebuilding new clusters it is necessary to check for this maximality and merge clusters where possible without violating the inconsistency condition.

Blocks World Example

Another brief example will illustrate this point. Here we want to learn the concept "something that is either yellow or spherical." For this, we use the following training instances from a blocks world.

(yellow pyramid soft large +)
(blue sphere soft small +)
(yellow pyramid hard small +)
(green large sphere hard +)
(yellow cube soft large +)
(blue cube soft small −)
(blue pyramid soft large −)

After the first three training examples have been given to the learner, the resultant description is the empty set.

```
concept_name:(yellow or spherical object)
 positive_part:
        cluster:description:()
                :examples:(yellow pyramid soft large)
                          (blue sphere soft small)
                          (yellow pyramid hard small)
 negative_part:
                :examples:_____________________
```

Since the next two examples are also positive, the only change to the above frame is the addition of the fourth and fifth training instances to the cluster examples. However, the sixth training instance is negative. This forces a split due to the inconsistency. In rebuilding the clusters this time, we rebuild starting with the last positive (fifth) example and work backwards as though the examples were put on a

stack. This is actually the order used in the original system. After the fifth and fourth examples are processed, the following frame is produced.

```
concept_name:(yellow or spherical object)
 positive_part:
      cluster:description:(large)
           :examples:(yellow cube soft large)
                     (green large sphere hard)

 negative_part:
           :examples: _____
```

Next, when an attempt is made to merge the third example, an inconsistency results. Therefore, this example must be put into a separate cluster. The same applies when an attempt is made to merge the second example with either of the new clusters; merging with the first cluster results in the empty set, while merging with the second cluster results in the set (small) which is also inconsistent with the negative example (blue cube soft small). This forces the creation of a third cluster. Finally, after attempts are made to merge the first example (it merges with the first two clusters), we obtain the frame

```
concept_name:(yellow or spherical object)
 positive_part:
      cluster:description:(large)
           :examples:(yellow cube soft large)
                     :(green large sphere hard)
                     :(yellow pyramid soft large)
      cluster:description:(yellow pyramid)
           :examples:(yellow pyramid hard small)
                     (yellow pyramid soft large)
      cluster:description:(blue sphere soft small)
           :examples:(blue sphere soft small)
negative_part:
           :examples:(blue cube soft small)
```

Note that we still have not arrived at the target concept. The last training instance, a negative one,

(blue pyramid soft large −)

is needed to do the trick. The first cluster is inconsistent with this instance. Therefore, it must be split. After completing this split we get the new frame

```
concept_name:(yellow or spherical object)
 positive_part:
      cluster:description:(yellow soft large)
           :examples:(yellow cube soft large)
                     (yellow pyramid soft large)
      cluster:description:(green large sphere hard)
           :examples:(green large sphere hard)
      cluster:description:(yellow pyramid)
           :examples:(yellow pyramid hard small)
                     (yellow pyramid soft large)
      cluster:description:(blue sphere soft small)
           :examples:(blue sphere soft small)
 negative_part:
           :examples:(blue cube soft small)
                     (blue pyramid soft large)
```

All of the new clusters are now consistent with the negative examples. But the clusters are not maximal, since it is possible to merge some clusters without violating the inconsistency condition. To obtain maximality, the clusters must be rewritten and merged where possible. This rewrite is accomplished by copying the first cluster and then successively merging or copying the other clusters in combination. Of course, a merge can be completed only when an inconsistency does not result.

The first two clusters cannot be merged as we know from the above. The first and third clusters can be merged to give yellow. The second and fourth clusters can also be merged to produce sphere. These are the only merges that can be made that are compatible with both negative examples. The final frame then is given as

```
concept_name:(yellow or spherical object)
 positive_part:
      cluster:description:(yellow)
           :examples:(yellow cube soft large)
                     (yellow pyramid hard
                     small)
                     (yellow pyramid soft
                     large)
      cluster:description:(sphere)
           :examples:(green large sphere hard)
                     (blue sphere soft small)
 negative_part:
           :examples:(blue cube soft small)
                     (blue pyramid soft large)
```

It may have been noticed already by the astute reader that there is no reason why negative-part clusters could not be created as well. Allowing this more symmetric structure permits the creation of a broader range of concepts such as "neither yellow nor spherical" as well as the positive type of concepts created above. This is implemented by building clusters in the negative part of the frame using the negative examples in the same way as the positive examples. In building both descriptions concurrently, care must be taken to maintain consistency between the positive and negative parts. Each time a negative example is presented, it is added to the negative part of the model, and a check is made against each cluster in the positive part of the model for inconsistencies. Any of the clusters which are inconsistent are split into clusters which are maximal and consistent and which contain all the original examples among them. We leave the details as an exercise.

Network Representations

It is also possible to build clusters of network representation structures and to learn structural descriptions of objects. For example, the concept of an arch can be learned in a manner similar to the above examples. In this case our training examples could be represented as something like the following where, as in earlier chapters, ako means a kind of.

```
((nodes (a b c)
 (links  (ako a brick)
         (ako b brick)
         (ako c brick)
         (supports a c)
         (supports b c) +)
```

Since an arch can support materials other than a brick, another positive example of the concept arch might be identical to the one above except for the object supported, say a wedge. Thus, substituting (ako c wedge) for (ako c brick) above we get a second positive instance of arch. These two examples can now be generalized into a single cluster by simply dropping the differing conjunctive ako terms to get the following.

```
((nodes (a b c)
 links  (ako a brick)
        (ako b brick)
        (supports a c)
        (supports b c))
```

This is an over generalization. It can be corrected by a negative training example which uses some nonvalid object such as a sphere as the supported item.

```
((nodes (a b c)
 links  (ako a brick)
        (ako b brick)
        (ako c sphere)
        (supports a c)
        (supports b c) −)
```

This example satisfies the current description of an arch. However, it has caused an inconsistency. Therefore, the cluster must be split into a disjunctive description as was done before in the previous examples. The process is essentially the same except for the representation scheme.

In a similar manner the concept of uncle can be learned with instances presented and corresponding clusters created using a network representation as follows.

```
((nodes (a b c).
 links  (ako a person)
        (ako b person)
        (ako c person)
        (male c)
        (parent_of b a)
        (brother_of c b) +)
```

Again, we leave the remaining details as an exercise.

Before leaving these examples, the reader should consider what learning methods and tools have been used from the previous two sections, what types of bias have been used to limit the search, and what methods of generalization have been employed in the learning process.

18.7 SUMMARY

Inductive learning is accomplished through inductive inference, the process of inferring a common class from instances, reasoning from parts to the whole or from the individual to the universal. Learning a concept through induction requires generalization, a search through a lattice of hypotheses. Practical induction is based on the use of methods that constrain or direct the search process. This is made possible through the use of bias which is used to reduce the size of the hypothesis space or to impose preferential selection on the hypotheses.

A number of techniques are available for either selective or constructive generalization, including changing constants to variables, dropping conjunctive conditions, adding a disjunctive alternative, closing the interval, climbing a generalization tree, and generating chain properties, among others. Specialization is achieved through the inverse operation to generalization as well as some other methods like including exceptions.

An example of an inductive learning system was presented in some detail. This system constructs concept descriptions from positive and negative examples presented as attribute lists. The descriptions are created as clusters or disjuncts by first generalizing conjunctive descriptions from the positive training examples until an inconsistent negative example is experienced. This separates the clusters to produce compatible, disjunctive descriptions. The system can also learn structural descriptions in the form of network representations. It is possible to build negative disjunctive descriptions as well, building clusters in the negative part of the frame structure or both positive and negative descriptions.

EXERCISES

18.1. Define inductive learning and explain why we still use it even though it is not a "valid" form of learning.

18.2. What is the difference between a class and a concept?

18.3. What is the difference between selective, constructive, and expedient induction? Give examples of each.

18.4. What is the purpose of inductive bias?

18.5. Give three examples in which inductive bias can be applied to constrain search.

18.6. Use the following training examples to simulate learning the concept "green flower or skinny object." Build up the concept description in clusters using the same method as that described in Section 18.6.
(green tall fat flower +)
(skinny green short flower +)
(tall skinny green flower +)
(red skinny short weed +)
(green short fat weed −)
(tall green flower skinny +)

18.7. Work out an example of concept learning using network structures. The concept to be learned is the concept wife. Create both positive and negative training examples.

18.8. Write a computer program in LISP to build concept descriptions in the form of clusters like the examples of Section 18.7.

18.9. The method described in Section 18.7 for learning concepts depends on the order in which examples are presented. State what modifications would be required to make the learner build the same structures independent of the order in which training examples are presented.

18.10. Compare each of the generalization methods described in Section 18.4 and explain when each method would be appropriate to use.

18.11. Referring to the previous problem, rank the generalization methods by estimated computation time required to perform each.

18.12. Give an example of learning the negative of the concept "tall flower or red object," that is, something that is "neither a tall flower nor a red object."

19

Examples of Other Inductive Learners

19.1 INTRODUCTION

In this chapter we continue with our study of inductive learning. Here, we review four other important learning systems based on the inductive paradigms.

Perhaps the most significant difference among these systems is the type of knowledge representation scheme and the learning algorithms used. The first system we describe, ID3, constructs a discrimination tree for use in classifying objects. The second system, LEX, creates and refines heuristic rules for carrying out symbolic integrations. The third system, INDUCE, constructs descriptions in an extended form of predicate calculus. These descriptions are then used to classify objects such as soybean diseases. Our final system, Winston's Arch, forms conjunctive network structures similar to the ones described in the previous chapter.

19.2 THE ID3 SYSTEM

ID3 was developed in the late 1970s (Quinlan, 1983) to learn object classifications from labeled training examples. The basic algorithm is based on earlier research programs known as Concept Learner Systems or CLSs (Hunt et al., 1966). This

27–

system is also similar in many respects to the expert system architecture described in Section 15.3.

The CLS algorithms start with a set of training objects $O = \{o_1, o_2, \ldots, o_n\}$ from a universe U, where each object is described by a set of m attribute values. An attribute A_j having a small number of discrete values $a_{j1}, a_{j2}, \ldots, a_{jk}$ is selected and a tree node structure is formed to represent A_j. The node has k_j branches emanating from it where each branch corresponds to one of the a_{ji} values (Figure 19.1). The set of training objects O are then partitioned into at most k_j subsets based on the object's attribute values. The same procedure is then repeated recursively for each of these subsets using the other $m - 1$ attributes to form lower level nodes and branches. The process stops when all of the training objects have been subdivided into single class entities which become labeled leaf nodes of the tree.

The resulting discrimination tree or decision tree can then be used to classify new unknown objects given a description consisting of its m attribute values. The unknown is classified by moving down the learned tree branch by branch in concert with the values of the object's attributes until a leaf node is reached. The leaf node is labeled with the unknown's name or other identity.

ID3 is an implementation of the basic CLS algorithm with some modifications. In the ID3 system, a relatively small number of training examples are randomly selected from a large set of objects O through a window. Using these training examples, a preliminary discrimination tree is constructed. The tree is then tested by scanning all the objects in O to see if there are any exceptions to the tree. A new subset or window is formed using the original examples together with some of the exceptions found during the scan. This process is repeated until no exceptions are found. The resulting discrimination tree can then be used to classify new objects.

Another important difference introduced in ID3 is the way in which the attributes are ordered for use in the classification process. Attributes which discriminate best are selected for evaluation first. This requires computing an estimate of the expected information gain using all available attributes and then selecting the attribute having the largest expected gain. This attribute is assigned to the root node. The attribute having the next largest gain is assigned to the next level of nodes in the tree and so on until the leaves of the tree have been reached. An example will help to illustrate this process.

For simplicity, we assume here a single-class classification problem, one where all objects either belong to class C or U-C. Let h denote the fraction of objects

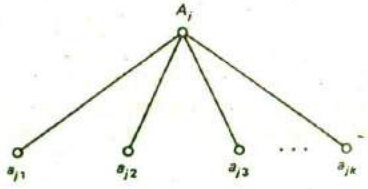

Figure 19.1 A node created for attribute A_j (color) with k discrete values, $a_{j1}, a_{j2}, \ldots, a_{jk}$ (red, orange, . . . , white).

that belong to class C in a sample of n objects from O; h is an estimate of the true fraction or probability of objects in U that belong to class C. Also let

c_{jk} = number of objects that belong to C and have value a_{jk}

d_{jk} = number of objects *not* belonging to C and having value a_{jk}

$p_{jk} = (c_{jk} + d_{jk}) / n$ be the fraction of objects with value a_{jk} (we assume objects have all attribute values so that $\Sigma_j p_{jk} = 1$)

$f_{jk} = c_{jk} / (c_{jk} + d_{jk})$, the fraction of objects in C with attribute value a_{jk}, and

$g_{jk} = 1 - f_{jk}$, the fraction of objects *not* in C with value a_{jk}.

Now define

$$H_c(h) = -h * \log_2 h - (1 - h) * \log_2(1 - h)$$

with $H_c(0) = 0$, and

$$H_{jk} = -f_{jk} * \log_2 f_{jk} - g_{jk} * \log_2 g_{jk}$$

as the information content for class C and attribute a_{jk} respectively. Then the expected value (mean) of H_{jk} is just

$$E(H_{jk}) = \Sigma p_{jk} * H_{jk}$$

We can now define the gain G_j for attribute A_j as

$$G_j = H_c - H_{jk}$$

Each G_j is computed and ranked. The ones having the largest values determine the order in which the corresponding attributes are selected in building the discrimination tree.

In the above equation, quantities computed as

$$H = -\Sigma p_i * \log_2 p_i \quad \text{with} \quad \Sigma_i p_i = 1$$

are known as the information theoretic entropy where p_i is the probability of occurrence of some event i. The quantities H provide a measure of the dispersion or surprise in the occurrence of a number of different events. The gains G_j measure the information to be gained in using a given attribute.

In using attributes which contain much information, one should expect that the size of the decision tree will be minimized in some sense, for example, in total number of nodes. Therefore, choosing those attributes which contain the largest gains will, in general, result in a smaller attribute set. This amounts to choosing those attributes which are more relevant in characterizing given classes of objects.

In concluding this section, an example of a small decision tree for objects described by four attributes is given in Figure 19.2. The attributes and their values are horned = {yes, no}, color = {black, brown, white, grey}, weight = {heavy, medium, light}, and height = {tall, short}. One training set for this example consists

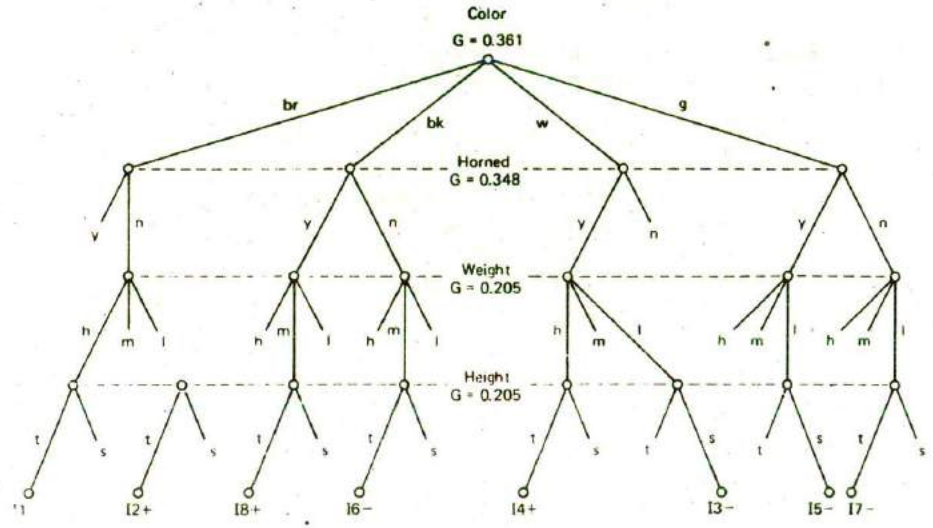

Figure 19.2 Discrimination tree for three attributes ordered by information gain. Abbreviations are br = brown, bk = black, w = white, g = gray, y = yes, n = no, h = heavy, m = medium, l = light, t = tall, and s = short.

of the eight instances given below where members of the class C have been labeled with + and nonmembers with − (Class C might, for example, be the class of cows).

I1 (brown heavy tall no −)
I2 (black heavy tall yes +)
I3 (white light short yes −)
I4 (white heavy tall yes +)
I5 (grey light short yes −)
I6 (black medium tall no −)
I7 (grey heavy tall no −)
I8 (black medium tall yes +)

The computations required to determine the gains are tabulated in Table 19.1. For example, to compute the gain for the attribute color, we first compute

$$H_c = -3 / 8 * \log_2 3 / 8 - 5 / 8 * \log_2 5 / 8 = 0.955$$

TABLE 19.1 SUMMARY OF COMPUTATIONS REQUIRED FOR THE GAIN VALUES.

		c_{jk}	$c_{jk} + d_{jk}$	f_{jk}	$\log f_{jk}$	$\log g_{jk}$	I_{jk}	G_j
$k = 1$	brown	0	1	0	—	—	0	
2	black	2	3	2/3	−0.585	−1.585	0.918	
3	white	1	2	1/2	−1.0	−1.0	1.0	0.361
4	grey	0	2	0	—	—	0	
$k = 1$	tall	3	6	1/2	−1.0	−1.0	1.0	
2	short	0	2	0	—	—	0	0.205
$k = 1$	heavy	2	4	1/2	−1.0	−1.0	1.0	
2	medium	1	2	1/2	−1.0	−1.0	1.0	0.205
3	light	0	2	0	—	—	0	
$k = 1$	yes	3	5	3/5	−0.737	−1.322	0.971	
2	no	0	3	0	—	—	0	0.348

The information content for each color value is then computed.

$$H(\text{brown}) = 0$$

$$H(\text{black}) = -2 / 3 * \log_2 2 / 3 - 1 / 3 * \log_2 1 / 3 = 0.918$$

$$H(\text{white}) = -1 / 2 * \log_2 1/ 2 - 1 / 2 * \log_2 1 / 2 = 1.0$$

$$H(\text{grey}) = 0$$

$$E(H_{\text{color}}) = 1 / 8 * 0 + 3 / 8 * 0.918 + 1 / 4 * 1.0 + 0 = 0.594.$$

Therefore, the gain for color is

$$G = H_c - H_{\text{color}} = 0.955 - 0.594 = 0.361$$

The other gain values for horned, weight, and height are computed in a similar manner.

19.3 THE LEX SYSTEM

LEX was developed during the early 1980s (Mitchell et al., 1983) to learn heuristic rules for the solution of symbolic integration problems. The system is given about 40 integration operators which are expressed in the form of rewrite rules. Some of the rules are shown in Figure 19.3a. Internal representations for some typical integral expressions are given in Figure 19.3b.

Each of the operators has preconditions which must be satisfied before it can be applied. For example, before OP6 can be applied, the general form of the integrand must be the product of two real functions, that is $udv = f_1(x) * f_2(x)dx$. Each operator also has associated with it the resultant states that can be produced by that operator. For example, OP6 can have